ASP.NET Core

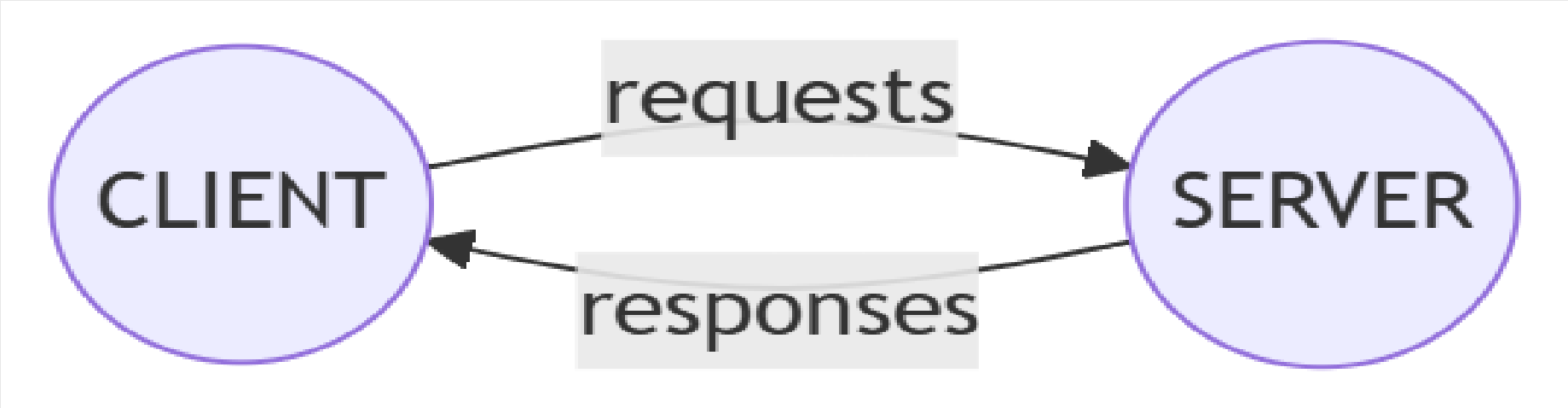
Prerequisite

1. Programming fundamentals using C#
2. OOP using C#
3. LINQ
4. EF Core
5. SQL Database fundamentals (MSSQL Server)
6. HTML, CSS, JS, JQuery

How Web Works?

- A user’s internet-connected device (like a computer, mobile, or tablet) that uses a web browser to access and interact with a web application.

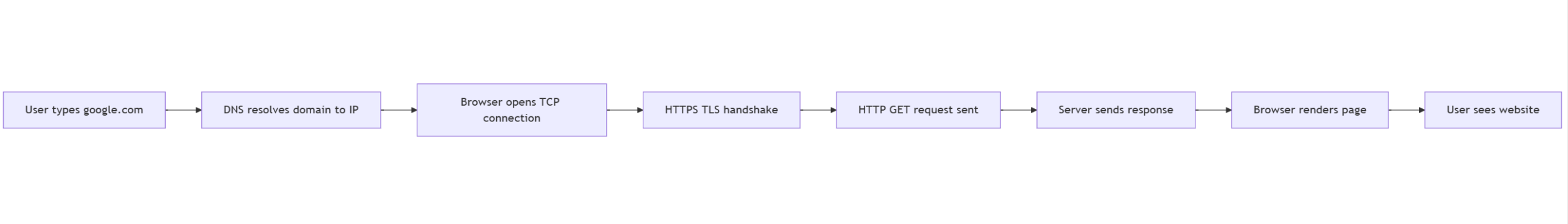
- Servers are computers that store the content of the page that the clients try to access.



How Web Works?

- Internet Connection
- TCP/IP
- DNS
- HTTP
- Files(Code - Assets)

How Web Works?



.NET History

1. NET Framework (2002)
   - First release by Microsoft
   - Windows-only
   - Still exists, but no new features
   - Maintenance & security updates only
   - Latest version 4.8

.NET History

2. .NET Core (2016)
   - Cross-platform (Windows, Linux, macOS)
   - Open-source
   - Faster, modular, lightweight
   - Introduced:
     - New ASP.NET Core
     - Better performance
     - Dependency Injection built-in
   - Latest version 3.1 (December 2019)

.NET Core Versions

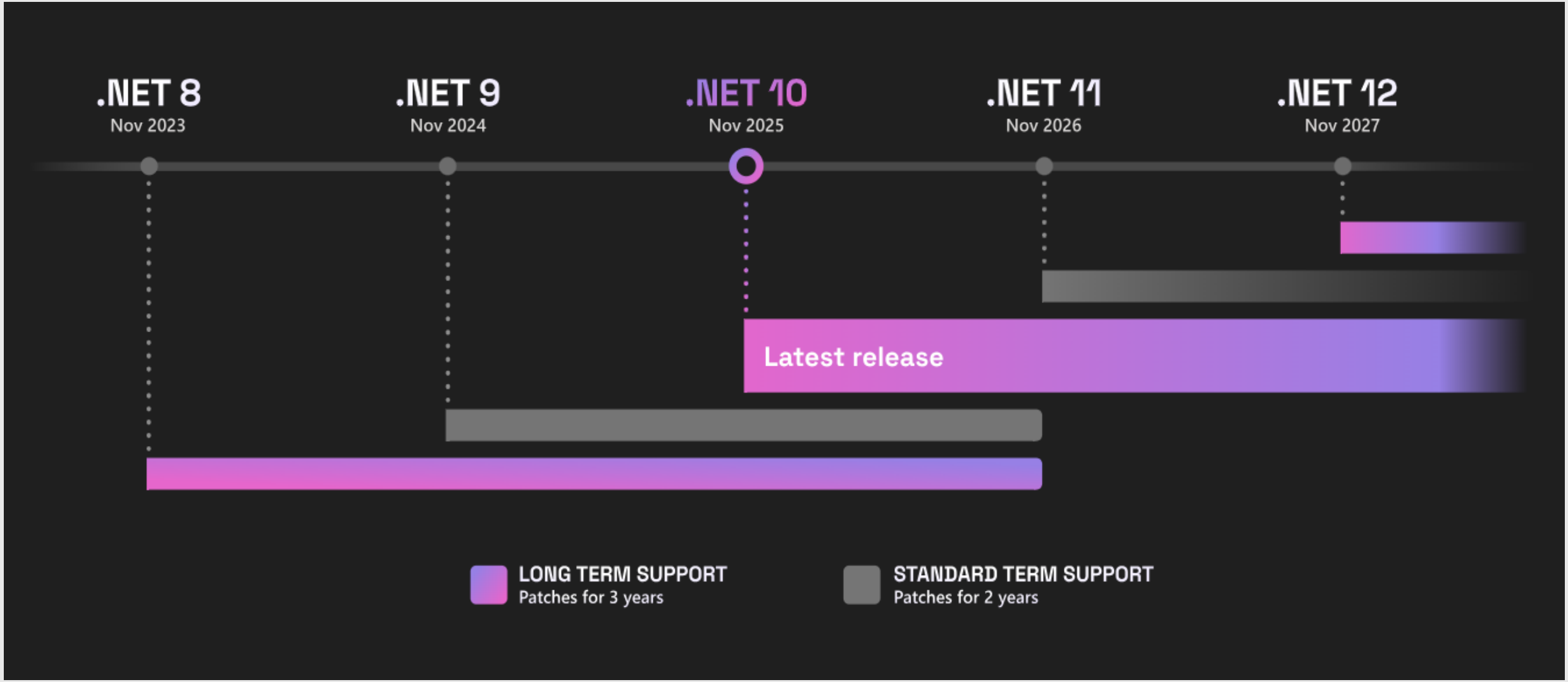
- .NET Core 1.0 (Initial Release – June 2016)
- .NET Core 1.1 (November 2016)
- .NET Core 2.0 (August 2017)
- .NET Core 2.1 (May 2018)
- .NET Core 2.2 (December 2018)
- .NET Core 2.2 (December 2018)
- .NET Core 3.1 (December 2019)

.NET History

3. .NET 5 (2020)
   - .NET Core and .NET Framework were merged into a unified platform called “.NET 5.”. Microsoft dropped the “Core” branding from .NET 5 onward to unify the .NET platforms.
   - One platform → .NET
   - No LTS, just a bridge version

.NET History

4. Modern .NET (.NET 6 → .NET 10)
   - Current
   - Cross-platform,
   - Cloud-first
   - LTS versions (6, 8, 10)
   - Used for:Web
     - APIs
     - Web Apps
     - Microservices
     - Desktop & Mobile (MAUI)


.NET 8
Nov 2023
.NET 9
Nov 2024
.NET 10
Nov 2025
.NET 11
Nov 2026
.NET 12
Nov 2027
Latest release
LONG TERM SUPPORT
Patches for 3 years
STANDARD TERM SUPPORT
Patches for 2 years

What is ASP.NET?

- ASP stands for Active Server Pages.
- Microsoft server-side scripting technology for building dynamic websites.
- ASP.NET is a web framework that runs on .NET for building modern web apps and services with .NET.
- ASP.NET extends the .NET platform with tools and libraries for building web apps and services

ASP.NET Versions

1. **ASP.NET (Old – .NET Framework)**
   - Windows only
   - Heavy IIS dependency
   - Web Forms & MVC
   - Slower, tightly coupled
- Examples:
   - ASP.NET Web Forms
   - ASP.NET MVC 5
   - ASP.NET Web API 2

ASP.NET Versions

2. **ASP.NET Core (Modern)**
   - Runs on **.NET Core/.NET**
   - Cross-platform
   - High performance
   - Built-in:
     - Dependency Injection
     - Middleware pipeline
     - Logging & Configuration

- Examples:
  - ASP.NET Core MVC
  - ASP.NET Core Web API
  - Minimal APIs
  - Blazor

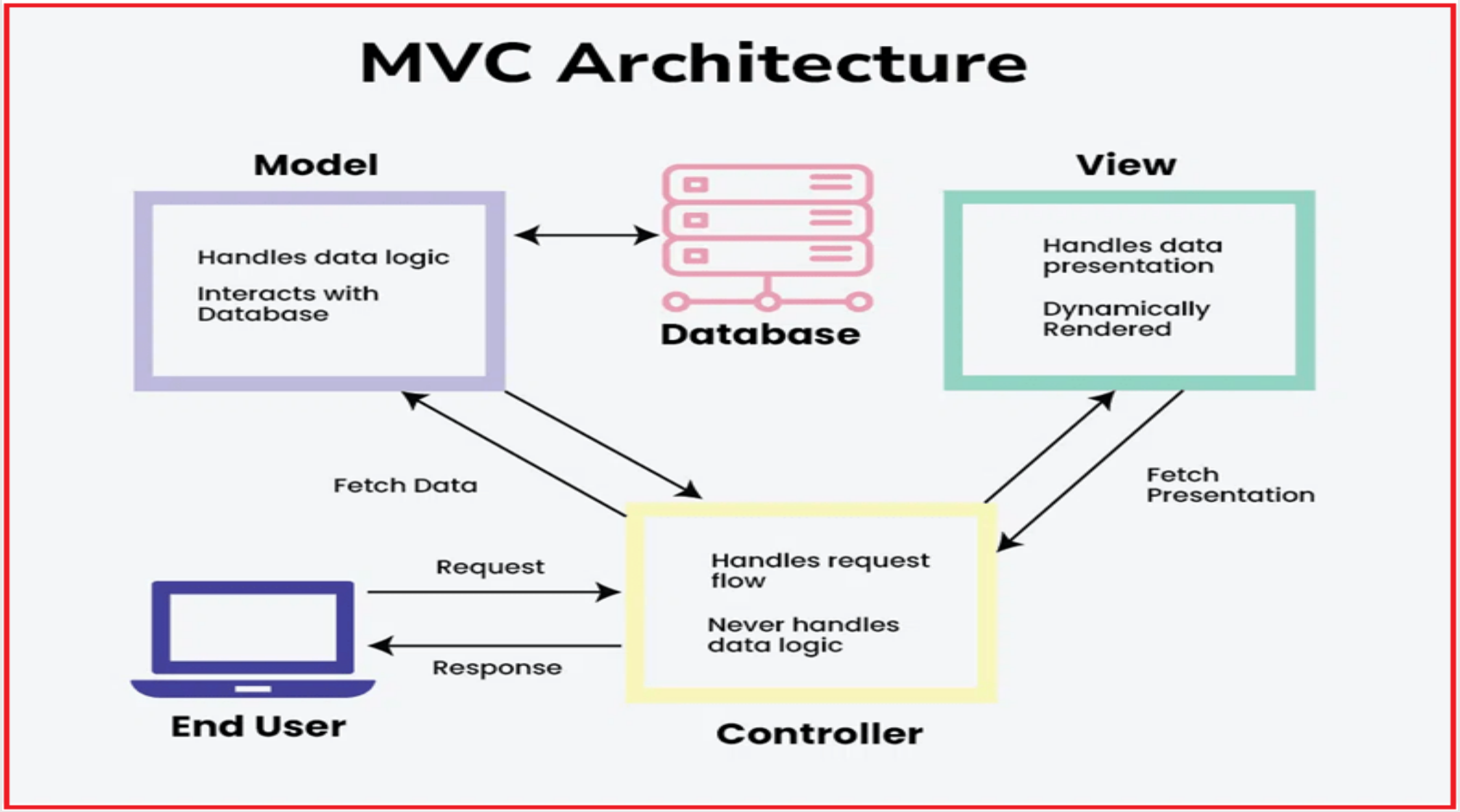
- Difference Between (Desktop Application – Web Application)
- Difference Between (Web Server – Web Application)
- Difference Between (Web Application – Web Service)

What is MVC?

- MVC stands for Model – View – Controller
- It’s a design pattern used to build applications (especially web apps) by separating responsibilities.

Part	Role
Model	The data + business rules
View	What the user sees
Controller	Handles requests & decides what to do

MVC Architecture



Why use MVC?

- Separation of concerns
- Easier maintenance
- Testable
- Perfect for large applications

MVC vs Web API

MVC	Web API
Returns HTML views	Returns JSON
Uses Razor	No UI
For web pages	For mobile / frontend apps
Controller	ControllerBase

MVC vs Web API

- Use MVC when:
  - Building server-rendered websites
  - Admin panels
  - Dashboards

- Use Web API when:
  - Frontend is React / Angular / Mobile app
  - You only need Raw data (JSON)

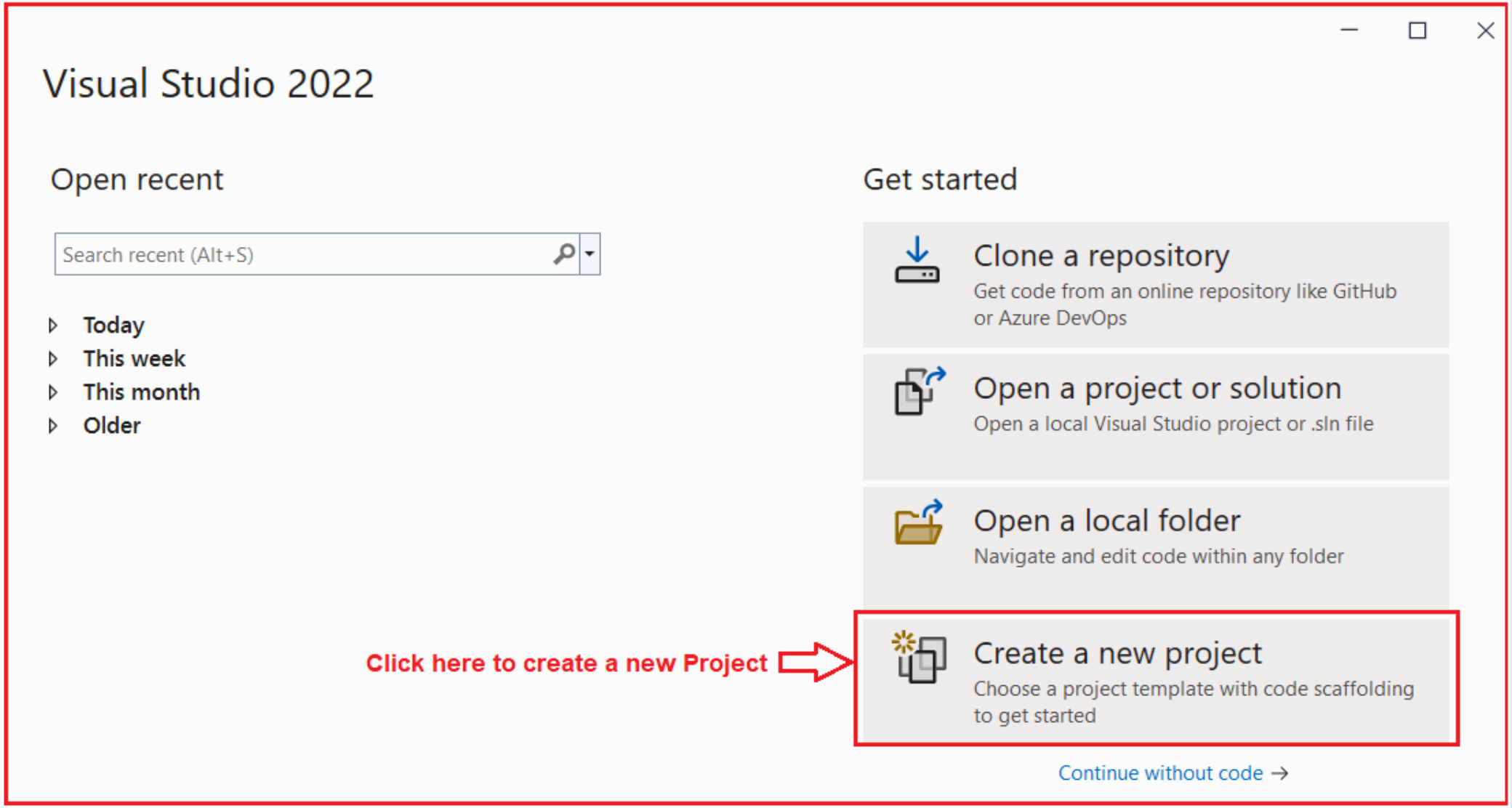
URL Mapping

- http://host:port/Employee/Get/1

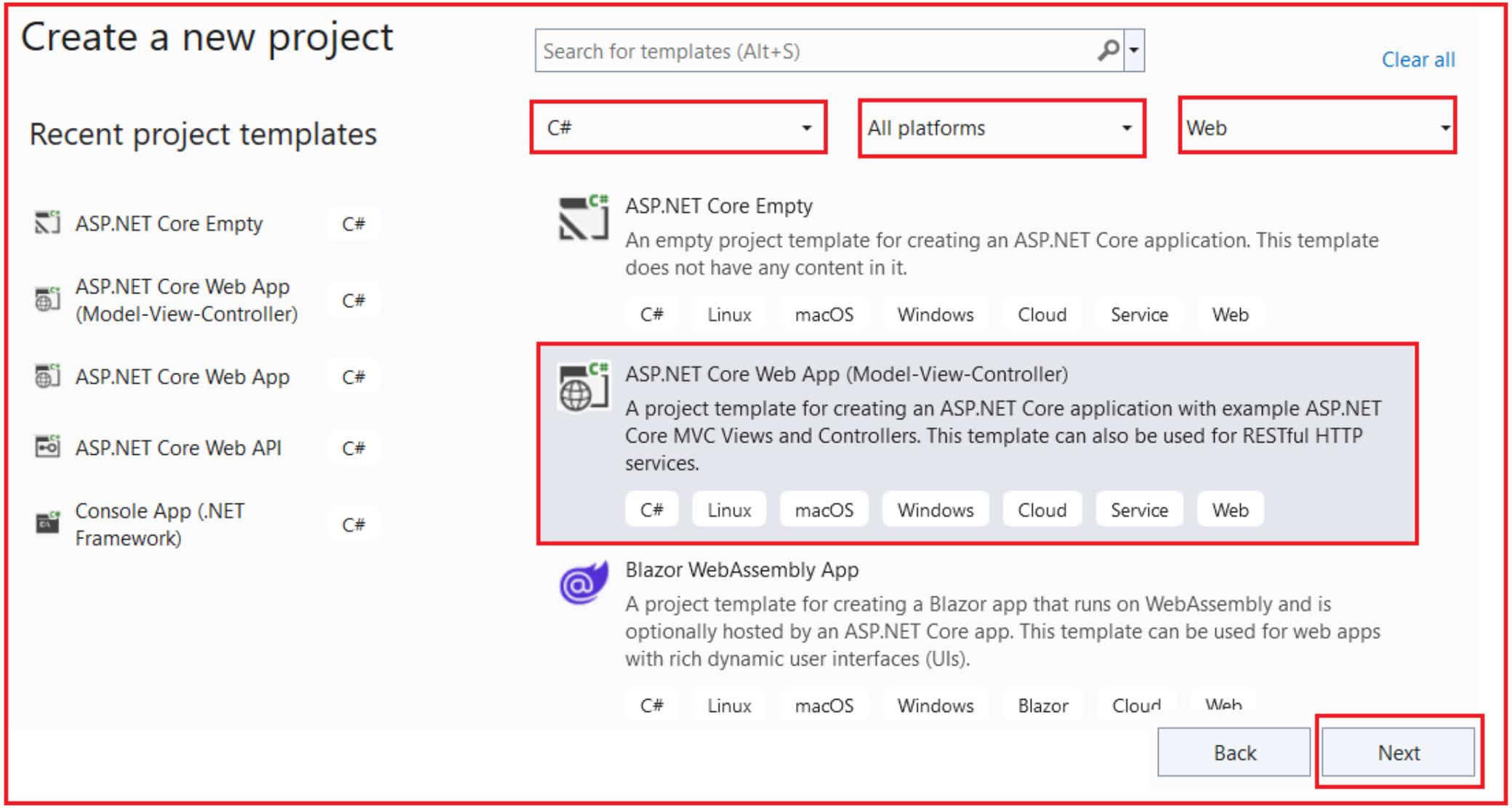
```
public class EmployeeController : Controller
{
    public IActionResult Get(int id)
    {
        // Retrieve the employee data from a database using the provided id
        return View();
    }
}
```

Creating an ASP.NET Core MVC Application

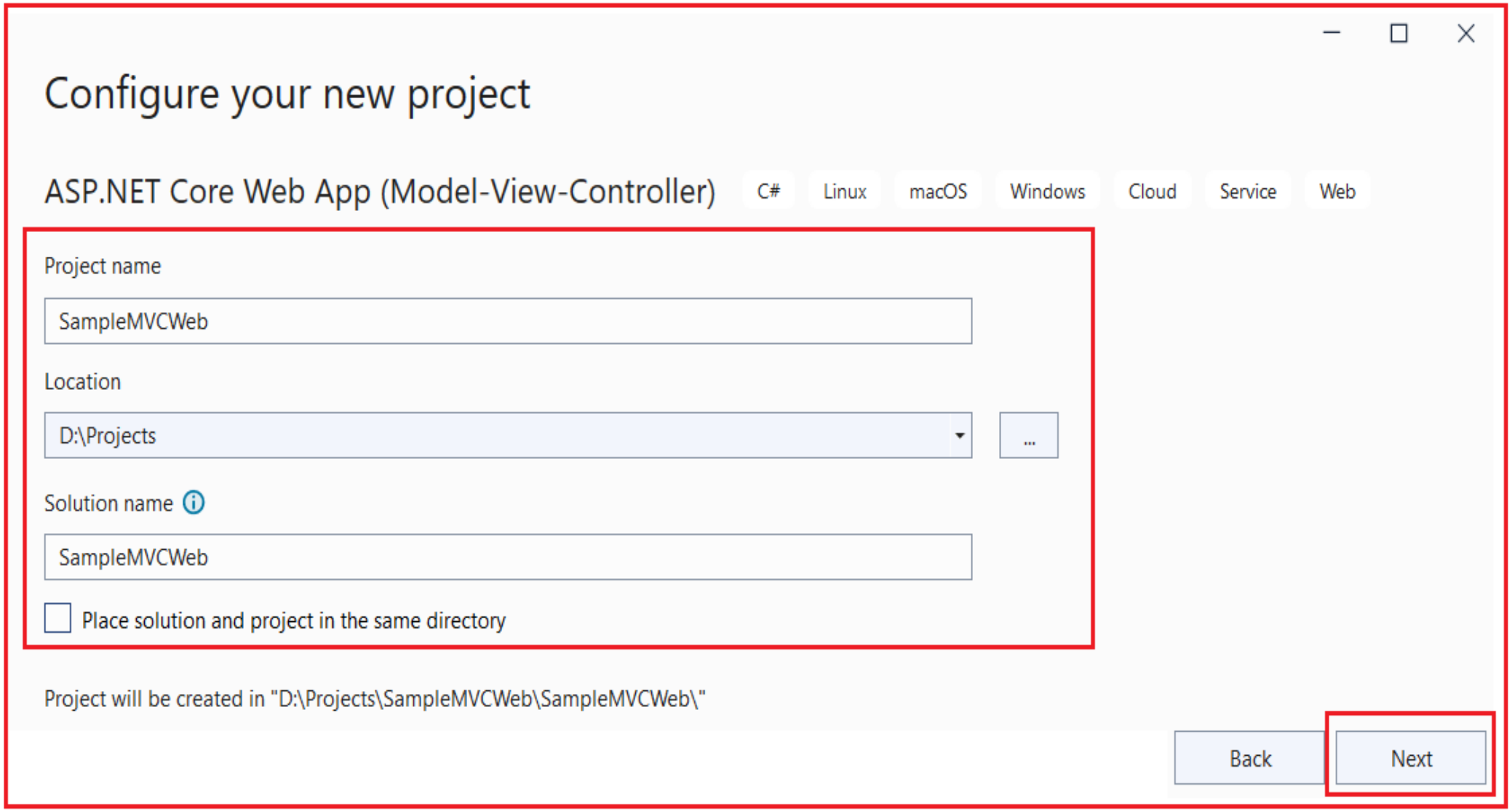
Creating an ASP.NET Core MVC Application



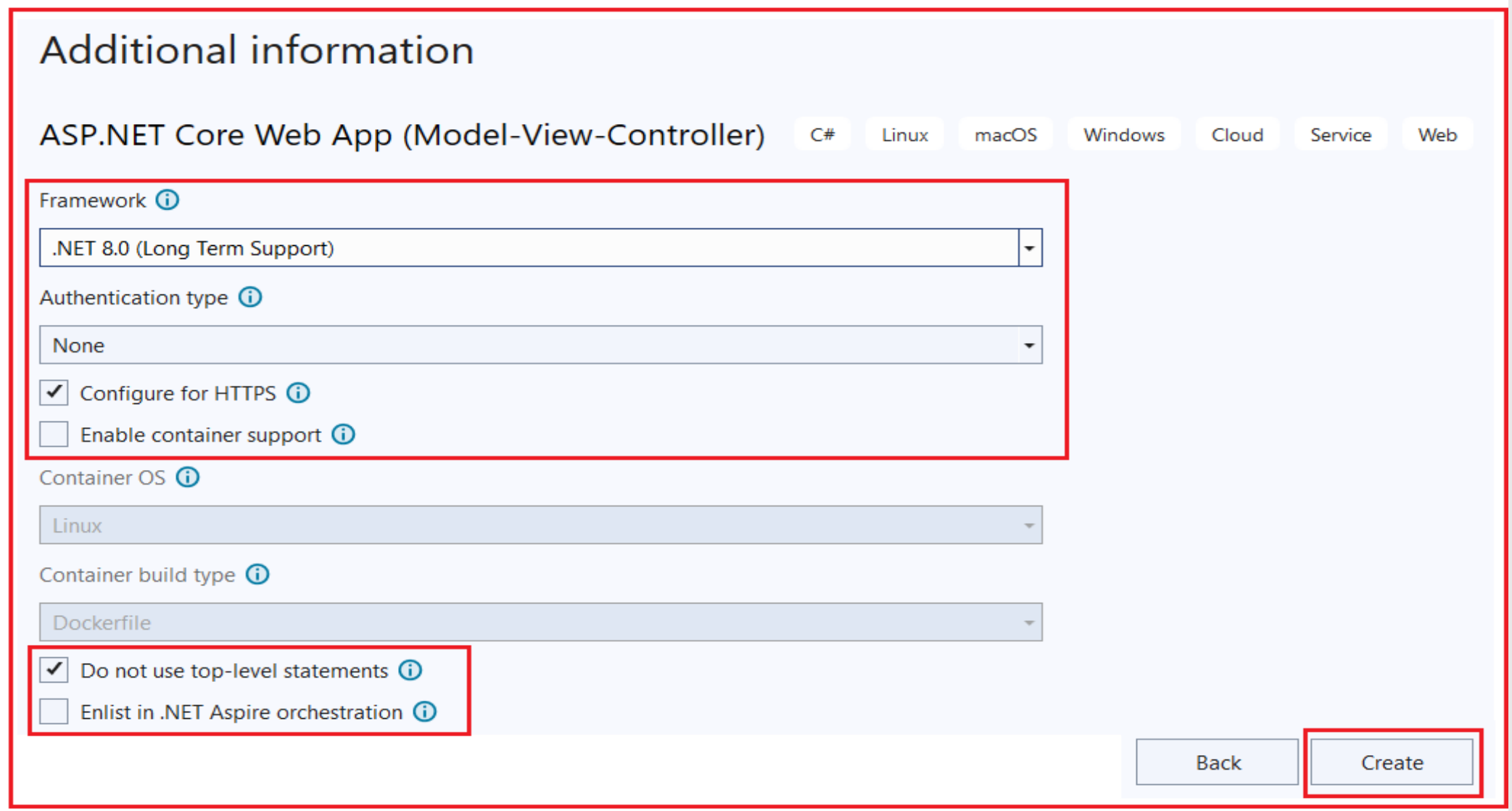
Creating an ASP.NET Core MVC Application



Creating an ASP.NET Core MVC Application



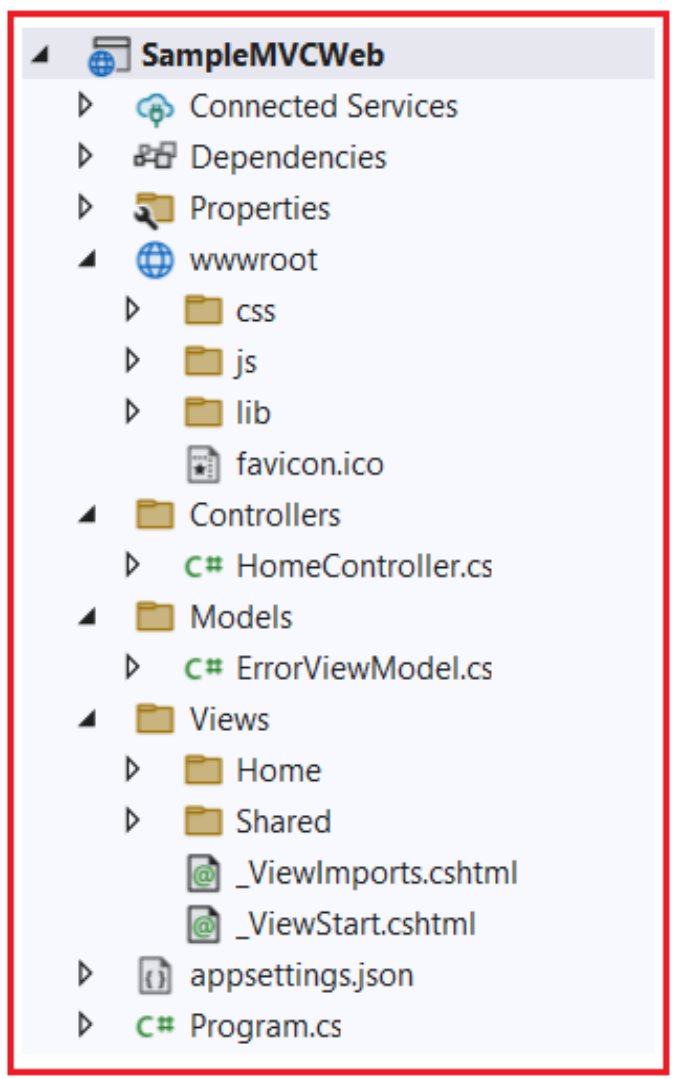
Creating an ASP.NET Core MVC Application



ASP.NET Core MVC Project File and Folder Structure

ASP.NET Core MVC Project File and Folder Structure

- Connected Services
- Dependencies Folder
- Properties Folder
- wwwroot Folder
- Controllers Folder
- Models Folder
- Views Folder
- appsettings.json File
- Program.cs File



Several key components work together in an ASP.NET Core MVC application to ensure the application runs smoothly.

Let’s discuss the roles and responsibilities of each folder and file of the ASP.NET Core MVC application.

ASP.NET Core MVC Project File and Folder Structure

- Connected Services:

  This section within the Solution Explorer in Visual Studio allows us to easily connect to external services such as Azure, Office 365, REST APIs, or third-party services. It helps integrate and configure APIs, SDKs, or services that your application depends on. It simplifies consuming external services by managing the necessary configurations and client code generation.

- Dependencies Folder

  The Dependencies folder contains the NuGet packages and other dependencies your project relies on. It includes:

  - **Packages:** The NuGet packages that your project uses.
  - **SDKs:** The .NET SDKs your project is targeting.
  - **Projects:** Other projects within the same solution that your project references.

ASP.NET Core MVC Project File and Folder Structure

- Properties Folder:

  This folder contains the launchSettings.json file, which is used to configure how the application is launched, including profiles for IIS Express and other environments. It includes settings for environment variables, application URL, and other settings related to how the application is run during development. You can specify different settings for different environments (like Development, Staging, and Production) in this file.

- wwwroot Folder:

  The wwwroot folder is the root folder for all static content your application uses, such as CSS, JavaScript, and image files. Files in this folder are accessible via a URL without additional configuration, making it ideal for storing assets directly needed in client-side interactions.

ASP.NET Core MVC Project File and Folder Structure

- Controllers Folder:

  The Controllers folder contains the controller classes. Controllers handle incoming HTTP requests, process them (often using services and models), and return the appropriate responses (usually views or JSON data) to the client. The methods inside controller classes are called actions and mapped to different routes defined in the application.

- Models Folder:

  The Models folder contains classes that represent the data of the application and logic to manage the data, i.e., how the data is going to be handled. Models represent the shape of the data and its business logic. This includes validation rules, business logic, and data-access logic. These classes are also used to pass data between the controllers and views. They often correspond to database tables in applications that use Entity Framework Core. For example, a Product model might represent a Product table in a database.

ASP.NET Core MVC Project File and Folder Structure

- Views Folder:

  The Views folder contains files to generate the HTML content returned to the client. It typically includes subfolders for each controller, containing the views associated with each action. These files are typically Razor view files (.cshtml), which mix HTML markup with C# code for dynamic content generation.

- Shared Folder:

  The Shared folder contains views and partial views that are shared across multiple other views, such as layout templates (e.g., _Layout.cshtml), error pages, or partial views used across various pages of the application. The Shared folder is typically located under the Views directory.

ASP.NET Core MVC Project File and Folder Structure

- _ViewStart.cshtml File:

  The _ViewStart.cshtml file is a special file in the Views folder that runs before every view in the application is executed. It is used to define common settings for all views, like specifying a common layout file (_Layout.cshtml) for the views. This makes it ideal for setting layout pages and other common settings.

- _ViewImports.cshtml File:

  The _ViewImports.cshtml file allows us to add common namespaces and Razor directives that we want to be available in all our views, such as @using declarations for namespaces, @inject for dependency injection, and @model declarations, to be available to all views without having to add them in each view individually. It helps to avoid repeating the same imports across multiple views.

ASP.NET Core MVC Project File and Folder Structure

- appsettings.json File:

  The appsettings.json file stores application configuration settings. It is a JSON file that includes settings such as connection strings, framework configuration settings, logging, and custom application settings. These settings can be loaded into the application at runtime and vary by environment.

- Program.cs File:

  The Program.cs file is the application’s entry point. It sets up the web host that will host the app, configures services, logging, configuration sources, and the HTTP server, and configures the ASP.NET Core application pipeline and services.

ASP.NET Core MVC
Data Passing Techniques

Data Passing Techniques in MVC

- Different Ways to Pass Data to Views in ASP.NET Core MVC:
  - ViewData: A dictionary object (ViewDataDictionary) that stores data as key-value pairs and passes it from Controller to View during the same request.
  - ViewBag: A dynamic wrapper around ViewData that allows property-like syntax instead of dictionary syntax.
  - Strongly Typed Models/View Models: Passing a model object directly from the Controller to the View using return View(model). This is the recommended and safest approach.
  - TempData: Used to pass data between two consecutive requests, such as during redirects.
  - Session: Stores data on the server side per user session. It can be accessed across multiple requests and can be read inside Controllers or Views.
  - Cookies: Stores data on the client side (browser), not server-side. Cookies are sent with each request and can be accessed in Controllers or Views.

ViewData / ViewBag

- There are scenarios where you want to send a **small amount of temporary data** from the Controller to the View that is **not part of the model**.

  In such cases:
  - **ViewData** and **ViewBag** are useful for transferring temporary data from Controller to View.
  - They work only during the **same HTTP request**.
  - They should be used for small, non-critical data.

  Common Real Examples
  - Page title
  - Success / error message (without redirect)
  - Dropdown list data
  - UI flags (e.g., IsAdmin = true)
  - Small helper values

ViewData in ASP.NET Core MVC Application

- **ViewData** is a dictionary object of type **ViewDataDictionary** that allows us to pass data from a Controller to a View using key-value pairs.
  ViewData is request-scoped, meaning the data it contains is only available during the current HTTP request. It is useful for passing small, non-complex pieces of information.
  - It stores data as string keys and object values.
  - It is accessible within both the Controller and the View.
  - The data in **ViewData** only lasts for the duration of the current HTTP request.

ViewData in ASP.NET Core MVC Application

- If you go to the definition of ViewData, you will see that it is a property in the Controller base class, and its type is ViewDataDictionary. This property allows us to attach data to our controller, which is then available for use in the corresponding view. This property is also decorated with the ViewDataDictionary attribute.

```
[ViewDataDictionary]
public ViewDataDictionary ViewData { get; set; }
```

- ViewDataDictionary class implements the IDictionary<string, object?> interface. Since it stores data as objects, explicit type casting is required when retrieving data in the View.

```
public class ViewDataDictionary : IDictionary<string, object?>
{

}
```

How to use ViewData in MVC Application?

- Using **ViewData** is straightforward:
  - In the Controller, assign data to the ViewData dictionary using a string key.
  - In the View, retrieve the data by accessing the ViewData dictionary with the same key and casting it to the appropriate type.

```
ViewData["Title"] = " Student Details" ;
```

Controller

```
@ ViewData["Title"]
```

View

```
Student student = new Student()
{
    StudentId = 101,
    Name = "James",
    Branch = "CSE",
    Section = "A",
    Gender = "Male"
};

ViewData["Student"] = student;
```

Controller

```
@{
    var student = ViewData["Student"]
        as FirstCoreMVCApplication.Models.Student;
}
```

View

As the data stored in the ViewData dictionary is of Student Type so we need to explicitly cast it to Student Type

When to Use ViewData in ASP.NET Core MVC?

ViewData is ideal in the following situations:

- When you need to pass **small pieces of data** from the Controller to the View that don't belong to your main model, such as status messages, page titles, or other metadata.
- When you want to quickly share data between a Controller and its View **without creating a strongly typed model.**
- When the data is **temporary** and relevant only for the current request, it won't persist across multiple requests.
- When **type safety is not a major concern**, since data stored in ViewData must be explicitly cast to the correct type when accessed in the View.
- When you want to pass data to Views or Partial Views **without the overhead of defining a dedicated model class.**

However, for scenarios involving complex data or where maintainability and type safety are essential, using strongly typed Models is a better and more robust choice.

ViewBag in ASP.NET Core MVC Application

- **ViewBag** is a dynamic property of the Controller base class in ASP.NET Core MVC that allows you to pass data from a Controller to its corresponding View in a flexible and easy-to-use way. It allows us to attach data to our controller (ViewBag property of the controller), which is then available for use in the corresponding view.

  - **Dynamic Property:** You don’t need to define types or keys as strings. Instead, you can add new properties on the fly, like ViewBag.Message = “Hello World”.
  - **Wrapper Around ViewData:** Internally, ViewBag is implemented as a wrapper around the ViewData dictionary, so both share the same underlying data. This means anything you add to ViewBag can be accessed through ViewData and vice versa.
  - **Scope:** Just like ViewData, ViewBag data lasts only for the duration of the current HTTP request. It is not persisted between multiple requests.

What does “dynamic” mean in ViewBag?

The dynamic type was introduced in **C# 4.0**. It behaves somewhat like the var keyword in that it can hold any type of value, but with a key difference: the actual type is resolved **at runtime** rather than compile time. This provides flexibility but sacrifices compile-time type checking.

How to Access Data from ViewBag in MVC

You can set and access a string (or other simple types) on ViewBag directly. You don't need to cast or use any string key; just reference the property name you set.

Important Notes About ViewBag

- **Dynamic, Not Type-Safe:** ViewBag does not provide compile-time type checking or IntelliSense support. If you misspell a property or try to access a property that wasn't set, you will only know at runtime (can result in a NullReferenceException).
- **Scope:** Data in ViewBag is only valid for the current request. It cannot be used for redirects or persist between requests.
- **Great for Small, Temporary Data:** Ideal for passing simple data, such as messages, titles, flags, or a small object, to a view or layout.
- **Not for Complex Apps:** For large applications or when passing lists or complex data, it is better to use strongly typed models for safety and maintainability.

Differences Between ViewBag and ViewData in MVC

- Though both ViewBag and ViewData serve the same purpose, passing data from Controller to View, they differ mainly in how you access and use them:

  Type:
  - ViewData is a dictionary (ViewDataDictionary) accessed via string keys.
  - ViewBag is a dynamic object that allows property-like access.

  Syntax:
  - ViewData requires string keys and casting when retrieving data.
  - ViewBag offers simpler syntax without the need for casting.

  Usage:
  - ViewData is more complex and requires explicit casting.
  - ViewBag is more concise and easier to use, especially for small data.

  Internals:
  - ViewBag is a **wrapper around ViewData**, so data stored in one can be accessed by the other.
  - If you store data using ViewBag, you can access it using ViewData.
  - Similarly, if you store the data using ViewData, you can still access the data using ViewBag.

  Type Safety:
  - Neither is type-safe, but ViewBag is more readable due to its dynamic nature.

ViewData vs ViewBag in MVC

When to Use ViewData

- ViewData is useful when you need to work with data in a dictionary-like structure, especially in scenarios involving loops or conditional logic where dictionary operations (like checking for keys, adding, or removing entries) are more straightforward.
- Since ViewData is a ViewDataDictionary, you can use methods like .ContainsKey(), .Remove(), or .Keys that can be handy in more dynamic data scenarios.
- Use ViewData when you might want to pass data in a way that requires explicit control over keys or when you prefer to treat the data as a collection.

When to Use ViewBag

- ViewBag offers cleaner, more readable syntax using dynamic properties, so it’s ideal when you want quick and simple data passing without needing explicit keys or dictionary operations.
- It is best suited for passing small amounts of simple data (e.g., page titles, messages, flags) where type checking is not critical and you prefer avoiding casting.
- ViewBag is excellent when you want to keep your Controller and View code concise and easy to write and read.

Strongly Typed View in ASP.NET Core MVC

- **A Strongly Typed View** is a Razor view that is bound to a specific data model (class). This means the view directly knows about the shape and type of data it should expect, allowing for full IntelliSense support, compile-time type checking, and easy access to model properties.

  - The View declares the type of model it expects via the @model directive at the top of the Razor view.
  - This means you can directly access the model’s properties without casting within the view.
  - Strongly typed Views improve code clarity, maintainability, and reduce runtime errors.

Why Do We Need a Strongly Typed View in MVC?

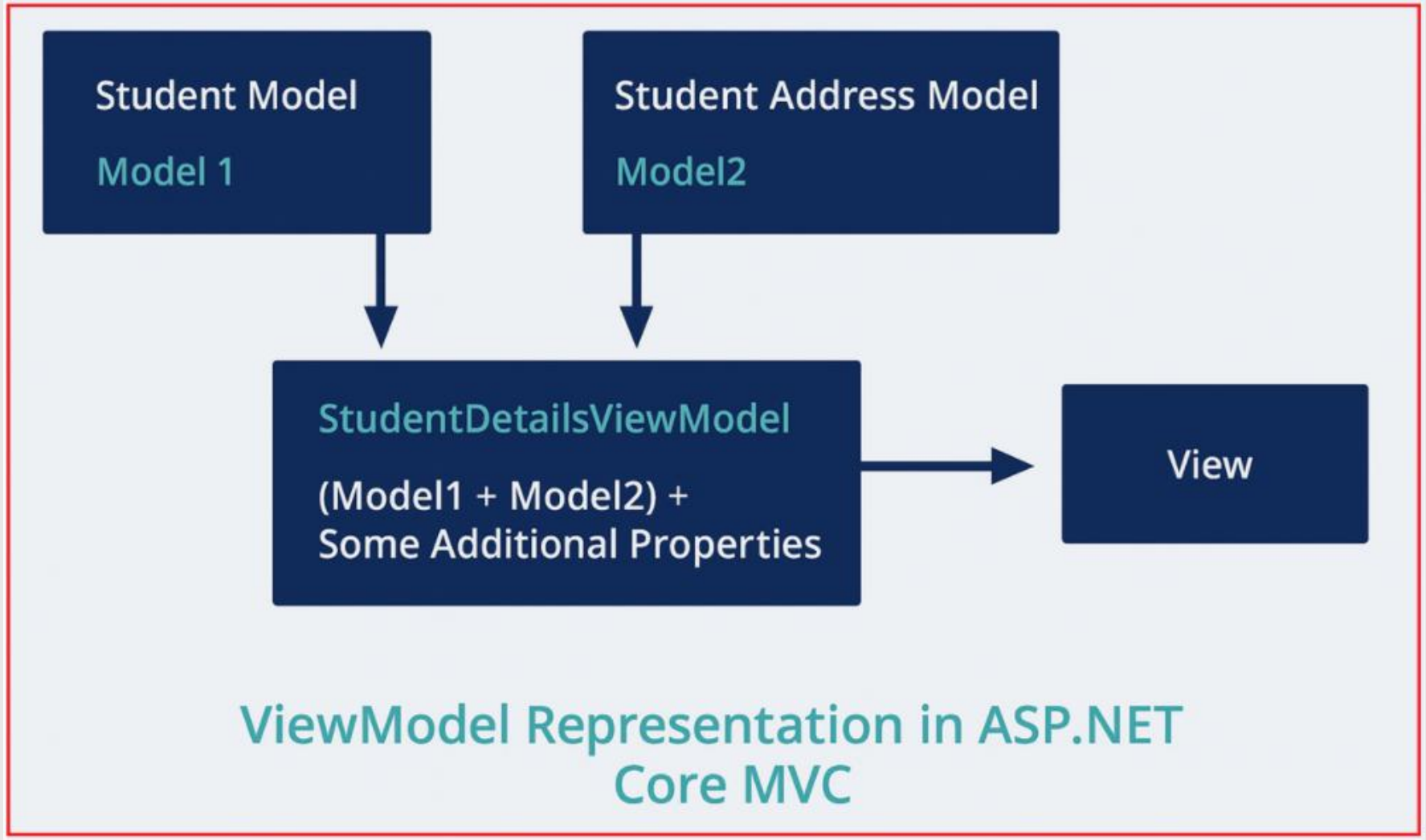
Using strongly typed Views offers several advantages:

- **Type Safety:** The compiler checks that you access properties that actually exist on the model, preventing typos and invalid property access. Therefore, errors such as typos or incorrect types are caught at compile time, not at runtime.
- **IntelliSense Support:** Visual Studio (and other editors) display model properties and methods, which speeds up development and reduces errors.
- **Refactoring Safety:** If you change your model (e.g., renaming a property), the compiler will alert you to update all related views.
- **Eliminates Casting:** No need to cast data from object to the expected type as you do with ViewData or ViewBag.
- **Separation of Concerns:** It enforces a clear contract between the Controller and View about what data is expected.

ViewModel in ASP.NET Core MVC

- **A ViewModel** is a special class explicitly created to hold the data and UI state required by a particular View. It differs from the Entity or Domain Model (which represents your business logic or database structure) because:
  - A ViewModel is designed exclusively for the **presentation layer**, i.e., View.
  - It can combine properties from **multiple domain models.**
  - It can include **calculated fields, UI state flags**, and any additional information the View needs.
  - It acts as a **Data Transfer Object (DTO)** between the Controller and View, containing exactly the data needed, no more, no less.
  - ViewModels **do not represent database tables** but are designed to suit the View's requirements.

ViewModel in ASP.NET Core MVC



Why Do We Need a ViewModel in MVC?

There are several reasons why ViewModels are essential:

- **Combining Multiple Models:** Views often require data from several sources. Since strongly typed views accept only one model, ViewModels consolidate multiple models into a single object.
- **Customizing Data for the View:** ViewModels can reshape, format, or aggregate data specifically for the UI. For example, you could add a property to the ViewModel that formats the student's full address as a single string.
- **Separation of Concerns:** By keeping your UI models (ViewModels) separate from your business/domain models, you make your code cleaner and easier to maintain. The ViewModel reflects only the **presentation** needs, not your entire business logic.
- **Adding UI State:** ViewModels can include UI-related information, such as dropdown lists, checkbox states, or validation messages, that are not present in the domain model.
- **Simplifies Validation:** You can apply validation attributes (such as [Required] and [EmailAddress]) to ViewModel properties for user input scenarios, without adding those UI concerns to your domain models.

HTTP Protocol

What is HTTP?

HTTP stands for HyperText Transfer Protocol

- Application-layer protocol
- Used for communication between:
  - Client (Browser / Mobile App)
  - Server (Web Server / API)
- Foundation of the World Wide Web

How HTTP Works?

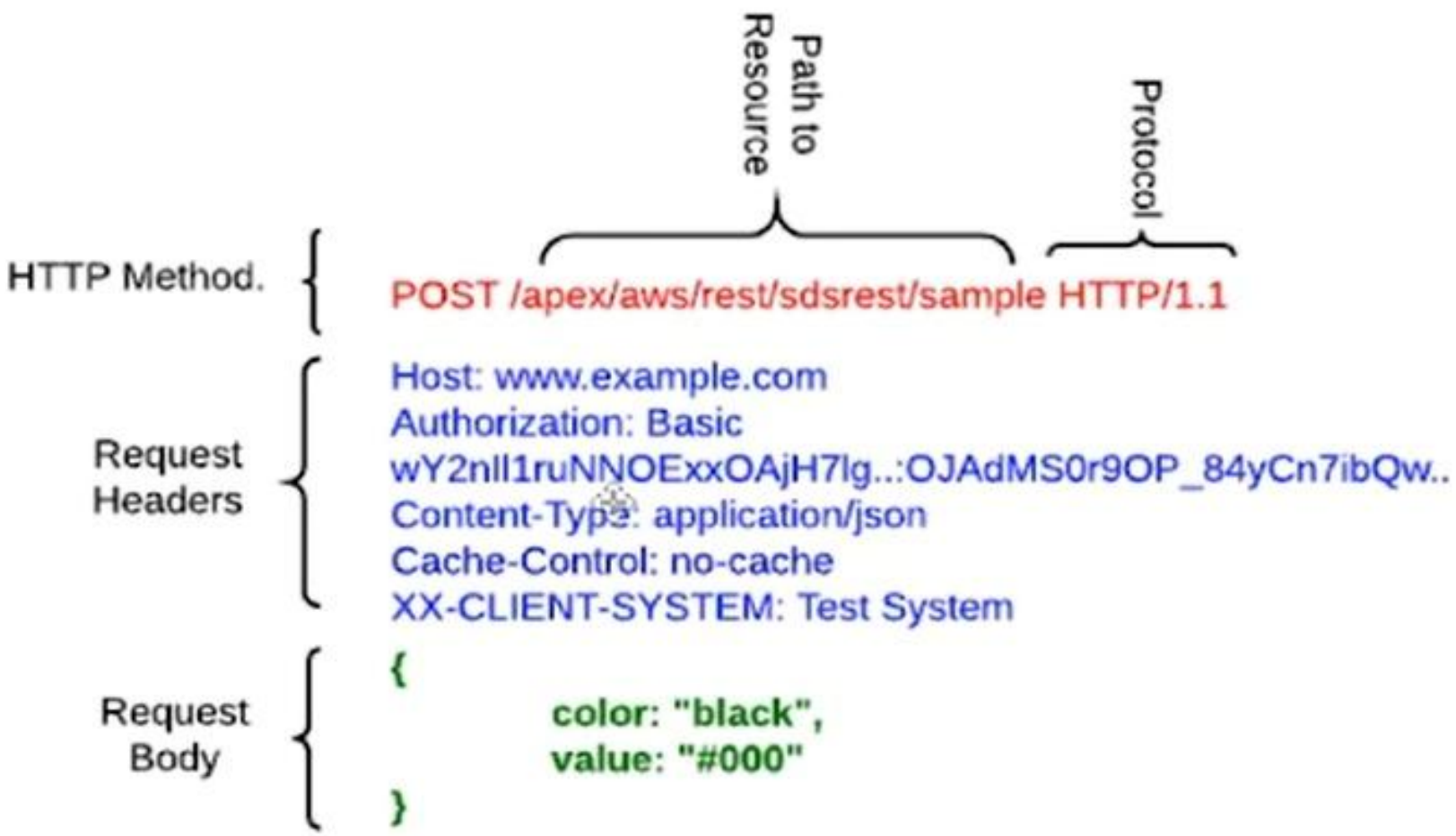
HTTP follows a **Request** → **Response** model:

- Client sends an HTTP Request
- Server processes it
- Server sends an HTTP Response

Example flow:

Browser → Request → Server
Server → Response → Browser

How HTTP Works?



Structure of HTTP Request

An HTTP Request contains:

- Method (GET, POST, etc.)
- URL
- Headers
- Body (optional)

Structure of HTTP Response

An HTTP Response contains:

- Status Code (200, 404, etc.)
- Headers
- Body (data)

What Are HTTP Verbs?

HTTP Verbs define **what action the client wants to perform** on a resource.
Main verbs:

- GET
- POST
- PUT
- PATCH
- DELETE

Get Method - Used to retrieve data

Request	Response
URL	Response Status - Status Code
Verb/Method	Content-Type (Application/Json – Application/xhtml)
-	Body (Content)

Post Method - Used to create new resource

Request	Response
URL	Response Status - Status Code
Verb/Method	Content-Type (Application/Json – Application/xhtml)
Content-Type (Application/Json – Application/xhtml)	Body (Content) (Optional)
Body (Content)	-

Put Method - Used to update resource

Request	Response
URL	Response Status - Status Code
Verb/Method	Content-Type (Application/Json – Application/xhtml)
Content-Type (Application/Json – Application/xhtml)	Body (Content) (Optional)
Body (Content)	-

Delete Method - Used to Delete resource

Request	Response
URL	Response Status - Status Code
Verb/Method	Content-Type (Application/Json – Application/xhtml)
-	Body (Content) (Optional)

HTTP Status Codes

- Identify the type of request success and failures
  - 1xx – Informational Response
  - 2xx – Successful Response
    - 200 – Ok
    - 201 – Created
  - 3xx – Redirect Response
    - 301 – Moved Permanently
  - 4xx – Client Error Response
    - 401 – Unauthorized
    - 404 – Not Found
  - 5xx – Server Error Response
    - 503 – Service Unavailable

HTTP Status Codes

Code	Name	Category	When to Use	Example Scenario
2xx	Success	—	The request was successful	—
200	OK	Success	Default success response	GET /users → Returns user list
201	Created	Success	Resource created successfully	POST /users → New user created
204	No Content	Success	Success but nothing to return	DELETE /users/123 → Resource deleted
3xx	Redirection	—	Further action needed	—
301	Moved Permanently	Redirection	Resource has a new permanent URL	Old endpoint redirects to new one
304	Not Modified	Redirection	Cached version is still valid	Browser cache is up-to-date
4xx	Client Errors	—	Client messed up	—
400	Bad Request	Client Error	Malformed request, invalid syntax	JSON body has wrong structure
401	Unauthorized	Client Error	Missing or wrong authentication	No API token provided
403	Forbidden	Client Error	Authenticated but no permission	User tries to delete admin account
404	Not Found	Client Error	Resource doesn't exist	GET /users/9999
409	Conflict	Client Error	State conflict (e.g., duplicate)	POST /users with existing email
422	Unprocessable Entity	Client Error	Valid request but semantic errors	Invalid age during user creation
429	Too Many Requests	Client Error	Rate limiting hit	API calls exceed quota
5xx	Server Errors	—	Server messed up	—
500	Internal Server Error	Server Error	Generic server failure	Unhandled exception
502	Bad Gateway	Server Error	Invalid response from upstream server	Proxy gets bad response
503	Service Unavailable	Server Error	Server temporarily down	Maintenance or overload
504	Gateway Timeout	Server Error	Upstream server timeout	Backend takes too long

HTTP vs HTTPS

HTTP → Not secure
HTTPS → Secure (uses SSL/TLS encryption)
HTTPS protects:

- Passwords
- Tokens
- Sensitive data

HTML Helpers in ASP.NET Core MVC

Microsoft Documentation

Dot Net Tutorials

What are HTML Helpers in ASP.NET Core MVC?

HTML Helpers in ASP.NET Core MVC are methods used in Razor views to generate HTML elements dynamically. They provide a way to render standard HTML elements programmatically, which helps reduce the amount of manual HTML code we have to write in our views.

Why HTML Helpers in ASP.NET Core MVC?

HTML Helpers in ASP.NET Core MVC are methods used in Razor views to generate HTML elements dynamically. They provide a way to render standard HTML HTML Helpers in ASP.NET Core MVC simplify the process of creating HTML elements on a web page and binding data to them. Using HTML Helpers reduces the chances of introducing typos or errors when manually writing HTML code. Additionally, HTML Helpers facilitate model binding, validation, and integration with ASP.NET Core features such as tag helpers, form validation, and data annotations. They also promote code reusability and maintainability, making views cleaner and easier to manage. programmatically, which helps reduce the amount of manual HTML code we have to write in our views.

For example:

Instead of:

```
<input type=”text” name=”firtsname” id=”firstname” />
```

Use:

```
@Html.TextBox(“firtsname”)
@Html.TextBox(“firtsname”, “Mohamed”)
@Html.TextBoxFor(e => e.FirstName)
```

At runtime, the above TextBox HTML helper method generates the following HTML

```
<input id=”firtsname” name=”firtsname” type=”text” value=”Mohamed” />
```

Types of HTML Helpers in ASP.NET Core MVC

HTML Helpers can be categorized into three types:

1. **Standard HTML Helpers:**

   These helpers generate basic HTML elements such as text boxes, checkboxes, radio buttons, etc.

   Examples include:

   - Html.TextBox()
   - Html.TextArea()
   - Html.DropDownList()

Types of HTML Helpers in ASP.NET Core MVC

2. **Strongly Typed HTML Helpers:**

   These helpers are associated with a specific data model and allow for compile-time checking of model properties. They use lambda expressions to refer to model properties directly.

   Examples include:

   - Html.TextBoxFor(model => model.PropertyName)
   - Html.EditorFor(model => model.PropertyName)
   - Html.DropDownListFor(model => model.PropertyName)

Types of HTML Helpers in ASP.NET Core MVC

3. **Custom HTML Helpers:**

   You can create custom HTML helpers by defining reusable methods to generate specific HTML markup. These helpers are typically used when you need custom functionality that the built-in helpers do not provide.

Categories of HTML Helpers in ASP.NET Core MVC

Category	Purpose	Common Methods	Generated HTML Example
Input Control Helpers	Generate HTML input elements for user data entry	• Html.TextBox() / For • Html.Password() / For • Html.CheckBox() / For • Html.RadioButton() / For • Html.DropDownList() / For • Html.Hidden() / For	<input type="text" /> <input type="checkbox" /> <input type="radio" />
Display Control Helpers	Render read-only model data	• Html.Display() • Html.DisplayFor() • Html.DisplayTextFor()	Plain text output
Label Helpers	Generate labels for model properties	• Html.Label() • Html.LabelFor()	<label>
Form Helpers	Create and manage HTML forms and form submission	• Html.BeginForm() • Html.EndForm() • Html.AntiForgeryToken()	<form> Hidden antiforgery input

Categories of HTML Helpers in ASP.NET Core MVC

Category	Purpose	Common Methods	Generated HTML Example
Validation Helpers	Show validation messages for model errors	• Html.ValidationMessageFor() • Html.ValidationMessage() • Html.ValidationSummary()	
Link Helpers	Generate hyperlinks within the application	• Html.ActionLink() • Html.RouteLink()	<a href="...">Link</a>
Editor Helpers	Automatically generate input fields based on model type	• Html.Editor() • Html.EditorFor()	Auto-generated input controls
Partial View Helpers	Render partial views inside a view	• Html.Partial() • Html.RenderPartial()	Rendered partial HTML
Model-Level Helpers	Work with entire model templates	• DisplayForModel() • EditorForModel()	Auto-generated UI for full model

Why HTML Helpers in ASP.NET Core MVC?

The primary reason to use HTML Helpers is to maintain a clean and maintainable codebase. HTML Helpers abstract the HTML generation part from the business logic, making the Razor views cleaner and more maintainable.

The following are the advantages of using HTML Helpers in ASP.NET Core MVC Applications:

- **Reusability:** Using HTML Helpers, we avoid duplicating HTML code across views, enhancing code reusability and maintainability.
- **Model Binding:** HTML Helpers simplify the connection between the view and model properties, ensuring that form fields are correctly bound to data properties. This makes it easier to work with forms and dynamic data.
- **Validation:** HTML Helpers automatically generate client-side validation for forms based on data annotations in the model, reducing the need to write manual validation code.
- **Consistency:** Helpers ensure consistent rendering of HTML elements across views, especially for form fields and controls.
- **Reduced Errors:** Since HTML Helpers are strongly typed (in the case of For helpers), they reduce the risk of errors by enabling compile-time checking of model properties.

Tag Helpers in ASP.NET Core MVC

Microsoft Documentation

Dot Net Tutorials

What are Tag Helpers in ASP.NET Core MVC?

Tag Helpers in ASP.NET Core MVC allow server-side code to participate in creating and rendering HTML elements in Razor views. They provide a way to add server-side functionality to HTML elements in a more readable way than traditional HTML helpers. Tag Helpers enhance HTML elements with attributes that generate dynamic HTML and allow developers to create cleaner and more maintainable markup.

The following are some of the Key Characteristics of Tag Helpers in ASP.NET Core MVC:

- **HTML-Like Syntax:** Tag Helpers use HTML-like syntax, which allows HTML developers to create and edit Razor views easily.
- **Model Binding:** Tag Helpers supports model binding.
- **Reusable:** They encapsulate the rendering logic and can be reused across different views, reducing code duplication.
- **Extensible:** Developers can create custom Tag Helpers to meet specific needs beyond what's provided by built-in Tag Helpers.
- **Easy to Understand:** Tag Helpers use HTML-like syntax, making it easier for developers to understand and work with than HTML helpers, which rely on method calls.
- **IntelliSense Support:** Provides strong IntelliSense support within Visual Studio, making development faster and reducing errors.

Types of Tag Helpers in ASP.NET Core MVC

Tag Helpers in ASP.NET Core MVC are classified into two Types. They are as follows:

- **Built-In Tag Helpers:**The ASP.NET Core framework provides these. They include functionality for common tasks such as form handling, link generation, and showing validation messages.

- **Custom Tag Helpers:** Developers can create their own Tag Helpers to encapsulate specific functionality or rendering logic required by their application.

Built-in Tag Helpers Type in ASP.NET Core MVC

The ASP.NET Core MVC Framework provides several built-in HTML Tag Helpers that generate the most common HTML elements and scenarios.

The following are some of the built-in HTML Tag Helpers available in ASP.NET Core MVC:

- **Form Tag Helpers:** <form> Tag Helper generates HTML forms. They help create forms that post data to actions in your controllers. For example: <form asp-controller="Account" asp-action="Login" method="post"> <!– form fields –> </form>
- **Input Tag Helpers:** <input>, <label>, <textarea>, <select>, etc, to facilitate form data management and validation. For example, <input asp-for="Name" /> and <input asp-for="IsAdmin" type="checkbox" />.
- **Anchor Tag Helper:** <a> Tag Helper for generating links using routing information. For example, <a asp-controller="Home" asp-action="Details" asp-route-id="1″>Details</a>
- **Image Tag Helper:** <img> Tag Helper for rendering images with dynamic sources. For example, <img src="~/images/pic.jpg" alt="Picture">
- **Environment Tag Helper:** <environment> Tag Helper for rendering content conditionally based on the current environment, such as Development, Staging, or Production.
- **Cache Tag Helper:** <cache> Tag Helper for output caching. This can help cache portions of your views to improve performance by reducing the need to recompute content on every request. For example, <cache> <!– Content to be cached –> </cache>
- **Partial Tag Helper:** <partial> Tag Helper for rendering partial views. This tag helper renders a partial view within another view.
- **View Component Tag Helper:** <view-component> tag helper renders a view component within a view. View components are similar to partial views but offer more encapsulation and flexibility.

Built-in Tag Helpers Type in ASP.NET Core MVC

Tag Helper	Purpose	Example Usage	Generated HTML Output
Form Tag Helper (<form>)	Generates HTML forms that post data to controller actions	<form asp-controller="Account" asp-action="Login" method="post">	<form action="/Account/Login" method="post">
Input Tag Helpers (<input>, <label>, <textarea>, <select>)	facilitate form data management and validation	<input asp-for="Name" /> <input asp-for="IsAdmin" type="checkbox" />	<input name="Name" value="..." />
Anchor Tag Helper (<a>)	Generates links using routing information	<a asp-controller="Home" asp-action="Details" asp-route-id="1">Details</a>	<a href="/Home/Details/1">Details</a>
Image Tag Helper (<img>)	Rendering images with dynamic sources	<img src="~/images/pic.jpg" alt="Picture" />	<img src="/images/pic.jpg" alt="Picture">
Environment Tag Helper (<environment>)	Renders content based on environment (Development, Production, etc.)	<environment include="Development">...</environment>	Content rendered only in specified environment
Cache Tag Helper (<cache>)	Caches portions of Razor views to improve performance	<cache> Cached content </cache>	Cached HTML output
Partial Tag Helper (<partial>)	Renders a partial view inside another view	<partial name="_LoginPartial" />	Rendered partial view HTML
View Component Tag Helper (<view-component>)	Renders a View Component inside a view	<view-component name="Menu" />	Rendered View Component HTML

Why Tag Helpers over HTML Helpers in MVC?

Tag Helpers are often preferred over HTML Helpers in ASP.NET Core MVC for several reasons:

- Readable and HTML-Like Syntax
- Better IntelliSense Support
- Cleaner and More Maintainable Code

Validation

What are Validations?

Validations in a web application refer to the processes and techniques used to ensure that the data entered by users is accurate, secure, and appropriate before it is processed or stored. Validations help maintain data integrity, enhance user experience, and secure the application from malicious inputs. That means the primary purposes of validations are:

- **Data Integrity:** Ensuring data is accurate and consistent.
- **User Experience:** Providing immediate and clear feedback to users.
- **Security:** Protecting the application from malicious inputs and attacks.

Types of Validation

- **Client-Side Validation:**
  - Validates data in the user's browser before submission, enhancing user experience by providing immediate feedback.
- **Server-Side Validation:**
  - Validates the data on the server, ensuring security and integrity.
- **Database Validation:**
  - Enforced at the database level, serving as the last line of defense for data integrity by defining constraints at the database level.

Client-Side Validation

Client-side validation uses JavaScript (or similar scripting languages) to validate user input in the browser before it's sent to the server. It enhances the user experience by providing immediate feedback, like notifying users of an invalid email format.

Benefits:

- **Immediate Feedback:** Provides immediate feedback without a server round trip. Users can correct errors immediately.
- **Reduced Server Load:** Early detection of errors minimizes the number of invalid requests sent to the server.
- **Improved User Experience:** Faster and more responsive forms lead to Improved User Experience.

Limitations:

- **Security Risks:** Users can bypass client-side validation by manipulating the client-side code.
- **Performance Issues:** Complex validations can slow down the browser, especially on less powerful devices.

Server-Side Validation

Server-side validation occurs on the server after the form is submitted. Unlike client-side validation, it is the most secure validation method for maintaining data integrity and security since it can't be bypassed. The following are the Key Points:

- Uses server-side languages like C#, Java, or Python.
- The user cannot bypass it.
- Acts as a backup for client-side validation.

- **Benefits:**
  - **Data Integrity:** Ensures data meets expected formats and constraints by validating the data types, formats, and rules.
  - **Security:** Prevents attacks like SQL injection, cross-site scripting (XSS), etc.
  - **Enforces Business Rules:** Validates more complex business rules that might not be possible on the client side.

Database Validation

Database Validations are the last line of defense to ensure data consistency across the system. They involve setting constraints directly within the database to prevent invalid data from being stored, such as:

- **Unique Constraints:** Ensuring uniqueness of values like usernames or email addresses. That is avoiding duplicate entries.
- **Foreign Keys:** Maintaining referential integrity between tables.
- **Check Constraints:** Enforce specific rules on data columns, such as age ranges.

Model Validation

- Data Annotation Validation
- Fluent API Validation

Data Annotations

What are Data Annotations in ASP.NET Core MVC?

- Data Annotations in ASP.NET Core MVC are attributes derived from the Attribute class. They define the behavior of model properties when interacting with views, databases, and validation processes. Data Annotations are widely used in Entity Framework Core, ASP.NET Core MVC, Web APIs, and Razor Pages applications, making them essential to model-driven development.

Data Annotations

Data Annotations Can be used for:

- **Validation:** Enforce business rules by specifying validation constraints on model properties.
- **Display Formatting:** Customize how data is displayed in the UI, including labels, formats, and order.
- **Database Mapping:** Influence how Entity Framework Core maps your model properties to database columns.

They are extensively used in:

- **ASP.NET Core MVC and Razor Pages:** For model validation and UI customization.
- **Entity Framework Core:** To configure the database schema during migrations.
- **Web API:** This is used to validate incoming data in API endpoints.

Model Validation using Data Annotation

Attribute	Purpose	Common Use Case
[Required]	Marks a field as mandatory	Username, Password
[MinLength(length)]	Sets minimum string length	Password min length
[MaxLength(length)]	Sets maximum string length	Name or title limit
[StringLength(maxLength, MinimumLength = minLength)]	Sets max and optional min length	Username length rules
[Range(min, max)]	Restricts numeric values to a range	Age between 18–60
[EmailAddress]	Validates email format	User email field
[RegularExpression("pattern")]	Validates against regex pattern	Password complexity
[Compare("OtherProperty")]	Compares two properties	Confirm password
[DataType(type)]	Specifies data type (UI hint)	Date, Time, Email
[Phone]	Validates phone number format	Mobile number
[Url]	Validates URL format	Website link
[CreditCard]	Validates credit card number	Payment info

Model Validation using Data Annotation

Note:
Data Annotations not only provide server-side validation but also trigger client-side validation by automatically generating HTML5 data attributes that integrate with jQuery Validation. This dual functionality helps create more responsive, better user experience and user-friendly forms by ensuring validation before server submission.

Display and Formatting Attributes using Data Annotation

Attribute	Purpose	Example Usage
[Display(Name = "Label")]	Specifies the label text shown for a field in the UI	Name = "First Name"
[DisplayOrder(order)]	Determines the order in which fields are rendered in the UI	DisplayOrder(1)
[DisplayName("DisplayName")]	A simpler alternative to [Display] for setting the display name	DisplayName("Email Address")
[Description]	Provides additional descriptive text, often used for tooltips or help text	Description("Enter a valid email")
[DisplayFormat(DataFormatString = "{0}")]	Controls how values (dates, numbers, etc.) are formatted in the UI	{0:yyyy-MM-dd}, {0:N2}
[DataType(DataType.Currency)]	Formats numeric values as currency	$1,234.00

Database Schema Configuration (Entity Framework Core)

Attribute	Purpose	Example Usage
[Key]	Marks a property as the **primary key** of the entity	public int Id { get; set; }
[ForeignKey("RelatedEntity")]	Specifies a **foreign key relationship** with another entity	ForeignKey("Order")
[DatabaseGenerated(DatabaseGeneratedOption.Identity)]	Indicates the **database auto-generates** the value (identity/auto-increment)	Identity column
[Column("ColumnNameInDb")]	Maps a property to a **specific database column name**	Column("user_name")
[Index]	Defines a **database index** on the column to improve query performance	Index(IsUnique = true)

Model Binding Attributes

Attribute	Purpose	Example Usage
[BindNever]	Prevents a property from being **included in model binding**, protecting it from being set via user input (e.g., IDs, timestamps)	public int Id { get; set; }
[BindRequired]	Ensures a property is **required during model binding**; adds a model state error if missing	public string Name { get; set; }

Why Use Data Annotation Attributes in MVC for Validation?

- **Simplifies Validation Logic:**
  Automatically validates data without writing extra code.

- **Dual Validation:**
  Ensure the same validation logic applies to the client and server.

- **Built-in Attributes:**
  Provides a wide range of built-in attributes for common validations like [Required], [Range], and [EmailAddress].

- **Custom Data Annotation:**
  We can also create Custom Data Annotation if Built-in Attributes do not fulfill our requirements.

- **Reduce Code Duplication:**
  Centralize validation logic within model classes, making it easier to manage and update.

When Not to Use Data Annotations

- **Complex Validation Logic:**
  Consider using Fluent Validation or custom validation if your validation rules involve complex logic or external dependencies. These frameworks provide more flexibility for handling complex business rules.

- **Separation of Concerns:**
  In applications that prefer strict separation between the domain model and presentation logic, avoid using Data Annotations directly in your domain classes. In such cases, external validation frameworks like Fluent API or separate metadata classes are better for keeping business logic separate from UI and persistence concerns.

- **Dynamic Validation Rules**:
  If validation rules need to change at runtime based on user roles or other conditions, hard-coded attributes may not be sufficient.

State Management

Microsoft Documentation

Dot Net Tutorials

What is State Management in MVC?

State management is how a web application remembers information about a user or a session across multiple HTTP requests.

Important: HTTP is **stateless**, meaning each request is independent. So ASP.NET MVC needs mechanisms to “remember” things like user login, shopping cart, or temporary data.

Why Do We Need State Management?

Examples of data that need to persist across requests:

- User login information
- Shopping cart items
- Form data between pages
- Temporary messages (e.g., “Your profile was updated”)

Without state management, the server **forgets everything** after responding.

Types of State Management

Server-Side State Management - Stores data on the server.

Method	Description
Session	Stores data per user session. Identified by a session ID (usually via cookie).
TempData	Temporary storage for data that persists only until the next request. Often used for messages after redirects.

Types of State Management

Client-Side State Management - Stores data **in the user's browser**.

Method	**Description**
Cookies	Small key-value pairs stored in the browser. Can persist across sessions.
Query Strings	Data sent in URL, e.g., /Products?category=2. Not secure for sensitive info.
Hidden Fields	Data stored in <input type="hidden"> in a form. Only sent back on POST.
LocalStorage / SessionStorage	Modern browser storage for JavaScript apps.

Sessions and Cookies in Web Applications

Why Do We Need Sessions and Cookies in Web Applications?

The core challenge in web development arises from the nature of the **HTTP protocol**. HTTP is a **stateless protocol**, which means that each request sent from a client (such as a web browser) to a server is completely independent of any other request. The server treats every request as new and unrelated, with no memory of previous interactions.
Because of this statelessness:

- The server **does not automatically know** whether two requests come from the same user, the same browser, or even the same device.
- Without additional mechanisms, every request is processed in isolation, making it impossible to maintain continuity or context across multiple requests.

This is where **sessions** and **cookies** become essential. For a better understanding, please refer to the following image.

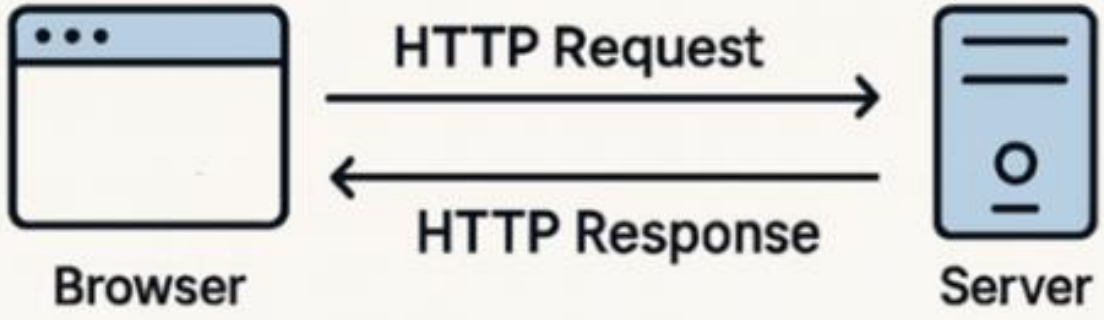
Why Do We Need Sessions and Cookies in Web Applications

Stateless Nature of HTTP

- Statelessness means every HTTP request stands alone
- No automatic tracking or memory

Role of Sessions and Cookies in State Management

- Sessions: These are server-side storage spaces
- Cookies: These are client-side storage spaces



Understanding the Stateless Nature of HTTP

- **Statelessness means every HTTP request stands alone:** When you visit a website, your browser sends an HTTP request to the server. Once the server responds, it forgets everything about that interaction. If you click a link or submit a form (which sends a new HTTP request), the server has no built-in way to know you are the same person who made the previous request.

- **No automatic tracking or memory:** Because of this, important user-specific data, like whether you are logged in, what items are in your shopping cart, or your site preferences, cannot be stored or retrieved by the server on its own across multiple requests.

The Role of Sessions and Cookies in State Management

To overcome HTTP's stateless limitation, web applications use **sessions** and **cookies** as tools to **remember information about users across multiple requests**. This enables web apps to behave more like traditional stateful applications.

- **Sessions:** These are server-side storage spaces where the server keeps user-specific data. Each session is identified by a unique session ID, which the server associates with a particular user.

- **Cookies:** These are small pieces of data stored on the client's browser. Cookies hold information such as the session ID or other key data, and are sent back to the server with every subsequent request, allowing the server to identify and link requests from the same user.

Real-time Use Cases of Sessions and Cookies in Web Applications

- **User Authentication Status**
- **User Preferences (e.g., Language, Theme)**
- **Shopping Cart Contents**
- **Form Data Temporarily Saved Between Pages**

Real-time Use Cases of Sessions and Cookies in Web Applications

User Authentication Status

- Whether the user is logged in or logged out is tracked using sessions or cookies.
- After a successful login, the server stores a session ID linked to the user's identity.
- The browser sends this session ID in cookies with each request, allowing the server to identify the user without requiring login credentials every time.
- This ensures secure access to personalized content, profile pages, or protected resources.

User Preferences (e.g., Language, Theme)

- Many websites allow users to choose their preferred language or display theme (such as dark mode or light mode).
- This preference is stored in cookies or sessions so that the user's settings persist across visits and page reloads.
- Without this, users would have to set their preferences on every new page or visit, resulting in a poor user experience.

Real-time Use Cases of Sessions and Cookies in Web Applications

Shopping Cart Contents

- In e-commerce applications, sessions or cookies store information about items users add to their shopping carts.
- This allows users to browse different pages or even leave and return later without losing their selected items.
- The server uses the session ID from the cookie to retrieve the cart contents from the session storage and display it accordingly.

Form Data Temporarily Saved Between Pages

- For multi-step forms or wizards, sessions can store form data temporarily as users navigate between pages.
- This prevents loss of information if users need to go back and forth between steps or if a page reload happens.
- It also helps pre-fill forms with previously entered data, reducing user effort and errors.

Cookies in ASP.NET Core MVC

What is a Cookie?

A cookie is a small piece of information that a web server sends to a user's web browser, which the browser then stores on the user's device (such as a computer, phone, or tablet). These cookies are stored as simple text data and consist of name-value pairs, such as a label and its associated value. Every time you revisit a website, your browser automatically sends these cookies back to the server with each subsequent request.

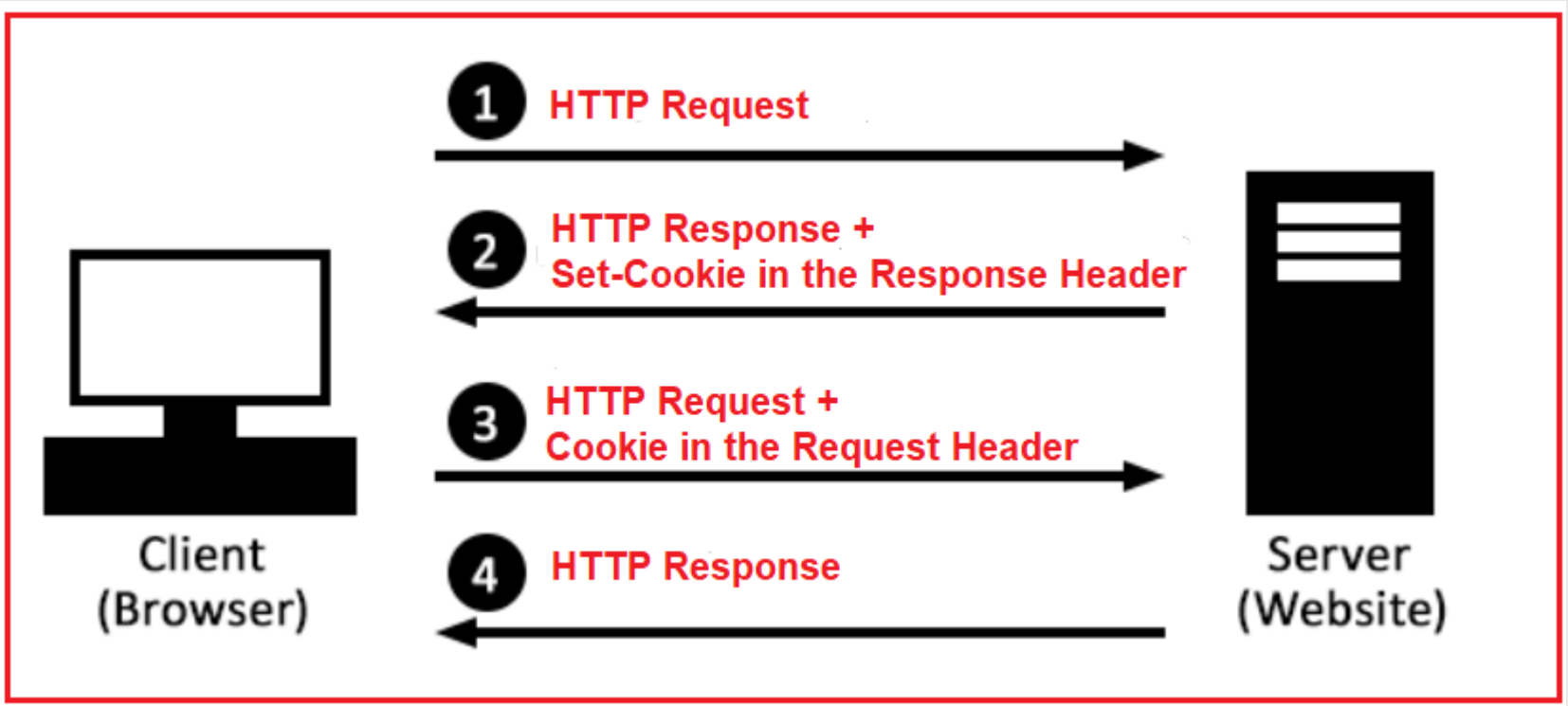
What Are Cookies Used For?

Cookies are commonly used to remember information about a user or their interactions with a website. This can include details such as:

- **User Identification:** For example, a user ID or username that enables the server to recognize the user making the request.
- **Authentication Tokens:** Saving login tokens allows users to avoid re-entering their credentials every time they visit the site.
- **User Preferences:** Settings like language choice, theme (dark or light mode), or font size to personalize the experience.
- **Session Management:** Keeping track of items in a shopping cart or the progress in a multi-step form.

How Do Cookies Work in Web Applications?

Cookies enable web applications to remember users and their preferences across multiple requests by exchanging small pieces of data between the client (browser) and the server. For a better understanding of how cookies work in a web application.



Types of Cookies in Web Applications

Cookies can be broadly categorized into **two main types** based on their lifespan and purpose:

- **Persistent Cookies**
- **Non-Persistent Cookies** (also known as session cookies).

What are Persistent Cookies in ASP.NET Core MVC?

Persistent cookies, also known as permanent or stored cookies, remain stored on the client's machine for a specified duration, even after the browser is closed. They persist across browser sessions unless explicitly deleted by the user or expired based on the set expiration time. These cookies are useful for remembering user preferences or login statuses across sessions.

Suppose you have a web application where users prefer to stay logged in for a longer period without re-entering their credentials every time they visit the site. In this case, a persistent cookie can store their authentication token.

Persistent Cookies in ASP.NET Core MVC

Persistent Cookies are Implemented in ASP.NET Core MVC Application by setting the Expires property in the CookieOptions object as follows.

```
var cookieOptions = new CookieOptions
{
    Expires = DateTime.Now.AddDays(30), // Expires in 30 days
    IsEssential = true, // Necessary for the application to function
    HttpOnly = true, // Accessible only by the server
    Secure = true // Only sent over HTTPS
};
Response.Cookies.Append("PersistentCookie", "CookieValue", cookieOptions);
```

What are Non-Persistent Cookies in ASP.NET Core MVC?

Non-persistent cookies, also known as session cookies, are temporary cookies stored only for the duration of the user’s session. They are deleted automatically when the user closes the browser. Unlike persistent cookies, which have a specific expiration date and remain stored on the client’s device between sessions, non-persistent cookies are deleted after the user’s browsing session.

Consider an online banking application where security is paramount. Non-persistent cookies store temporary data, such as session identifiers, and ensure that users are logged out when they close their browsers.

Non-Persistent Cookies in ASP.NET Core MVC

Non-persistent cookies are implemented in an ASP.NET Core MVC Application without specifying the Expires property in the CookieOptions object, as follows.

```
var cookieOptions = new CookieOptions
{
    IsEssential = true, // Necessary for the application to function
    HttpOnly = true, // Accessible only by the server
    Secure = true // Only sent over HTTPS
};
Response.Cookies.Append("NonPersistentCookie", "CookieValue", cookieOptions);
```

Differences Between Persistent vs Non-Persistent Cookies

In ASP.NET Core MVC, the primary difference between persistent and non-persistent cookies is their lifespan and how they are managed within a user's session.

- **Storage Duration:** Persistent cookies persist beyond the current session; non-persistent cookies are deleted when the browser session ends.

- **Purpose:** Persistent cookies are used for long-term storage of user preferences, login details, etc., while Non-Persistent cookies are used for temporary data like session identifiers.

- **Expiration Handling:** Persistent cookies have a specified expiration date and time; non-persistent cookies are deleted automatically when the browser session ends.

When Should We Use Persistent Cookies in MVC?

Use persistent cookies when you want to remember user preferences or login sessions across multiple visits. The following are some of the use cases of Persistent Cookies:

- **Remember Me Feature:** When users log in to a website, a “Remember Me” option is available to keep them logged in for an extended period. This is convenient for users who frequently return to your site.

- **User Preferences:** Storing user preferences like Themes, Language Settings, Notifications, etc of your website. Persistent cookies ensure that these preferences are remembered across different sessions.

- **Shopping Carts in E-commerce:** To remember the items in a user’s shopping cart across different sessions if they haven’t completed the checkout process.

When Should We Use Non-Persistent Cookies in MVC?

Use non-persistent cookies to store temporary data that should not persist beyond the current session. The following are some of the use cases of Non-Persistent Cookies:

- **Session Management:** Storing session identifiers for user authentication. The session ends once the user closes the browser.

- **Temporary data storage:** Temporary data storage that doesn’t need to persist beyond the current browsing session.

For example, a banking application can use non-persistent cookies to store session identifiers, enhancing security by ensuring users are logged out when they close their browsers.

Explanation of Important CookieOptions Properties

- **Domain:** Specifies the domain(s) where the cookie is valid. Setting this allows sharing cookies across subdomains if desired (e.g., .example.com makes it valid for sub.example.com too).
- **Expires:** A fixed expiration **date and time** when the cookie will be automatically deleted by the browser.
- **Path:** Defines the URL path that must exist in the requested URL for the cookie to be sent. Setting it to “/” means the cookie is sent on all requests within the entire domain.
- **Secure:** When true, cookies are only sent over **secure HTTPS connections**, protecting cookies from interception by attackers during transmission.
- **HttpOnly:** Prevents JavaScript running in the browser from accessing the cookie. This helps protect cookies from Cross-Site Scripting (XSS) attacks.
- **MaxAge:** Sets the duration (time span) the cookie remains valid from the moment it is created. It’s an alternative to specifying an exact expiration date.
- **IsEssential:** Indicates that the cookie is essential for the website to function properly, allowing it to be set even if the user has not consented to non-essential cookies (important for GDPR compliance).

Sessions in ASP.NET Core MVC

Why Do We Need Sessions in Web Applications?

The core challenge in web development arises from the HTTP protocol's stateless nature:

- Each HTTP request from the client (browser) to the server is independent.
- The server does not retain any memory or context from previous requests.
- This makes it impossible to automatically track a user's state, preferences, or ongoing interactions.

Sessions solve these problems by storing data on the server, with the browser carrying only a small session ID token (usually via a cookie). This approach is more secure and allows storing larger and more complex data compared to client-side cookies. This allows web apps to maintain data such as login status, shopping carts, user preferences, and other stateful information securely on the server.

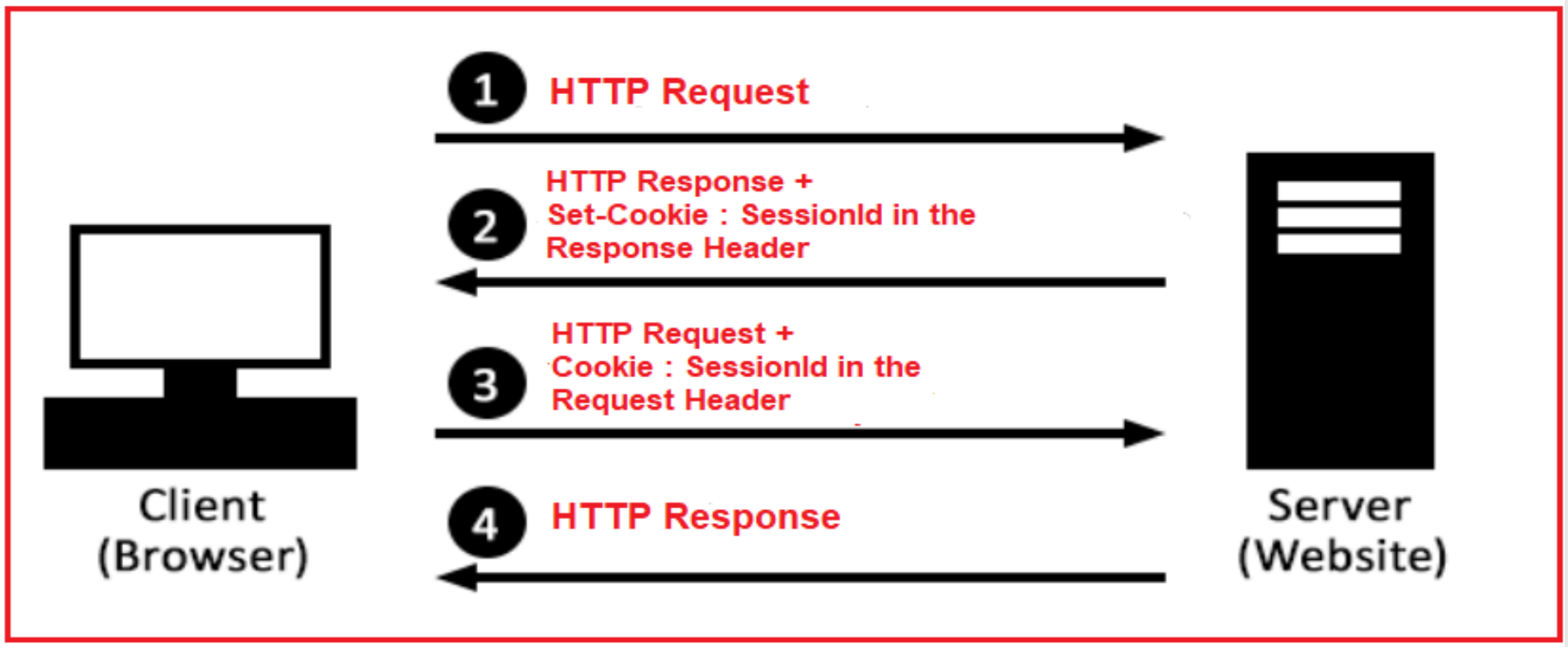
Real-Time Use Cases of Sessions in Web Applications

The following are some of the real-time scenarios:

- **User Authentication:** Track whether a user is logged in by storing authentication data in the session.
- **Shopping Cart:** Store cart items for e-commerce users between page visits and browsing sessions.
- **Multi-Step Forms Data:** Preserve multi-step form data temporarily as the user navigates pages.
- **User Preferences:** Save temporary preferences, such as theme selection, for the current session.
- **Temporary Data Storage:** Cache user-specific data that should not be persisted permanently.

How Does Sessions Work in Web Applications?

In web applications, a **session** is a mechanism for storing information about a user across multiple requests as they interact with a website. Since HTTP is stateless (meaning each request is independent), sessions enable the server to remember who the user is and maintain data such as login status, shopping cart contents, or preferences while the user browses the site.



Types of Sessions in Web Applications

There are mainly two ways to store session data in web applications:

In-Memory or In-Proc Sessions:

- This approach stores session data directly in the memory of the web server process handling the request. Because the data is in memory, access is very fast, making it suitable for smaller applications or those running on a single server.
- However, this method has limitations; if the server restarts or crashes, all session data is lost. Additionally, in environments with multiple servers behind a load balancer, each server has its own separate memory, resulting in session data not being shared across servers and leading to inconsistent user experiences.

Distributed or Out-Proc Sessions:

- To overcome the limitations of in-memory sessions, distributed session storage moves session data outside the web server process into external storage like databases or caching services (e.g., Redis, Memcached).
- This allows multiple servers to share the same session data, which is essential in load-balanced or cloud environments where requests can go to any server. Distributed sessions improve scalability, reliability, and fault tolerance since the session data persists independently of any one server’s lifecycle.

How To Use Sessions in ASP.NET Core?

ASP.NET Core MVC offers built-in support for managing sessions with simple configuration:

- **Enable Session Services and Middleware:** To add session support to your application, you need to register the session services and middleware, which handle reading and writing session cookies and managing session data.

- **Store and Retrieve Data:** ASP.NET Core sessions use a key-value storage model, allowing you to save and retrieve data using string keys. This is straightforward for managing user-specific data, such as shopping cart contents or login status.

- **Configure Session Options:** You can customize how sessions behave by configuring options such as the idle timeout (the duration a session remains active after inactivity), cookie properties (including security flags), and whether the cookie is essential for the application.

Session Storage Mechanisms in ASP.NET Core MVC

ASP.NET Core supports different ways to store session data depending on the deployment environment and requirements:

- **In-Memory Cache (default):**
  This is the simplest option where session data is stored in the server's memory. It's very fast and easy to use, but only suitable for single-server setups or development purposes, as data is lost when the server restarts or if the app scales horizontally.

- **Distributed Cache:**
  In production environments with multiple servers or when high availability is needed,a distributed cache is preferred. Session data is stored externally and shared by all servers. Common distributed cache providers include:
  - **Redis:** A fast, in-memory data structure store widely used for distributed caching.
  - **SQL Server:** Storing sessions in a database for persistence.
  - **NCache:** A commercial distributed cache for .NET applications.

Differences Between Cookies and Sessions in ASP.NET Core MVC

Differences Between Cookies and Sessions in MVC

- **Storage Location**
- **Lifespan (How Long Data Persists)**
- **Security**
- **Data Capacity**
- **Use Cases**
- **Accessibility**

Differences Between Cookies and Sessions

Storage Location

Cookies

- **Where?** Stored on the client's browser (user's device).
- **How?** When the server sends a cookie, it's included in the HTTP response. The browser saves this and automatically attaches it to all future requests to the same server.
- **Example:** If you set a cookie called Theme=Dark, it lives on the user's computer. On each request, the browser adds Theme=Dark to the HTTP headers.

Sessions

- **Where?** Stored on the server (e.g., in memory, database, or distributed cache).
- **How?** The server generates a unique Session ID and sends it to the client, typically as a cookie (like .AspNetCore.Session or any custom name). The actual data is kept safely on the server side.
- **Example:** If you store a shopping cart in the session, the browser only stores the session ID. All cart data stays on the server.

Differences Between Cookies and Sessions

Lifespan (How Long Data Persists)

Cookies

- **Persistent Cookies:** Remain on the user's device for a set time (e.g., 30 days), unless deleted. Example: "Remember Me" checkboxes often use persistent cookies.
- **Session Cookies:** Exist only while the browser is open; disappear when the user closes the browser.
- **Configurable:** Developers can set the expiration date and time when creating the cookie using attributes such as Expires or Max-Age.

Sessions

- **Default Lifespan:** Lasts as long as the session is active. Typically ends when the browser closes or after a configurable timeout period (e.g., 20 minutes of inactivity).
- **Configurable:** In ASP.NET Core, you can set the session timeout in the Program.cs (e.g., options.IdleTimeout = TimeSpan.FromMinutes(30);) where configuring the session.

Differences Between Cookies and Sessions

Security

Cookies

- **Risks:** Stored on the **client side**, cookies are vulnerable to client-side attacks like:
  - **Cross-Site Scripting (XSS):** Malicious scripts injected into the page can read cookies if not properly protected.
  - **Cross-Site Request Forgery (CSRF):** Exploiting cookies to perform unauthorized actions on behalf of users.
- **Protection: To improve cookie security, developers should use:**
  - **HttpOnly:** JavaScript can’t access these cookies.
  - **Secure:** Only sent over HTTPS.
  - **SameSite:** Restricts cross-site sending (mitigates CSRF attacks).
- **Encryption:** Sensitive data should be encrypted before being placed in a cookie.

Sessions

- **More Secure:** Only the session ID is exposed to the client. The actual data is safely stored on the server.
- **Server Control:** You can better control and audit access.
- **Still Needs Caution:** If someone hijacks a session ID (e.g., via session fixation), they could impersonate the user, but this is generally harder than stealing cookies.

Differences Between Cookies and Sessions

Data Capacity

Cookies

- **Limitations:** Browsers limit each cookie to around 4KB. Most browsers also limit the total number of cookies that can be stored per domain.
- **Overhead:** Every HTTP request includes cookies, and large numbers of cookies slow down requests.
- **Use:** Only small pieces of data (tokens, preferences, etc.).

Sessions

- **Capacity:** Can store much larger data (as long as server memory/database permits).
- **Efficiency:** Only the tiny session ID is sent with each request, minimizing actual data transfer.
- **Use:** Sessions allow for holding complex objects and larger data sets (e.g., shopping cart details, user preferences) without significantly affecting network performance.

Differences Between Cookies and Sessions

Use Cases

Cookies

- Best for small, non-sensitive data that must persist even after browser closure, like:
  - User preferences (theme, language).
  - Authentication tokens (JWT, with caution).
  - Remember Me flags.
- Useful when data needs to be accessed on both client (JavaScript) and server sides.

Sessions

- Ideal for:
  - Sensitive data that shouldn’t be exposed to the client.
  - Temporary user data that only needs to persist during a session, like:
    - Shopping cart contents.
    - Multi-page form state.
    - Login status is stored server-side.
- Since sessions require server resources, they are used when security or data size demands it.

Differences Between Cookies and Sessions

Accessibility

Cookies

- Accessible by both:
  - **Client side:** Via JavaScript (document.cookie).
  - **Server side:** Via the HTTP request (ASP.NET Core can read/write cookies).
- This dual accessibility makes cookies flexible but requires care for security.
- Suitable for data that needs to be read on both front-end and back-end, like showing a user's theme preference.

Sessions

- Only accessible on the server side.
- The client cannot read or modify session data directly.
- This reduces the risk of client-side tampering.

Considerations for Choosing Between Cookies and Sessions

Considerations for Choosing Between Cookies and Sessions

Choosing Between Cookies and Sessions

Data Sensitivity

Use sessions for senitiive information to reduce risk of exposure or tampering.

Data Size

Use sessions for large or complex data.

Persistence Need

Use cookies when data must persist across browser sessions orfter closing the browser.

Performance Impact

Since cookies sent with eververy HTTP request, large cookies can slow down requests and increase overhead

Scalability and Server Load

Sessions consume server memory/storage sinsce data is stored server-side; this can be a bottleneck for large-scale applications

Considerations for Choosing Between Cookies and Sessions

Data Sensitivity:

- Use **sessions** for sensitive information to reduce the risk of exposure or tampering.

Data Size:

- Use sessions for large or complex data.
- Cookies are only for small bits of data due to size constraints.

Persistence Need:

- Use **cookies** when data must persist across browser sessions or after the browser is closed.
- Sessions expire with inactivity or browser closure.

Considerations for Choosing Between Cookies and Sessions

Performance Impact:

- Since cookies are sent with every HTTP request, large cookies can slow down requests and increase bandwidth.
- Sessions send only the session ID, reducing overhead on requests.

Scalability and Server Load:

- Sessions consume server memory and storage since data is stored on the server; this can be a bottleneck for large-scale applications.
- Cookies offload storage to the client, reducing server memory usage.

TempData in ASP.NET Core MVC

What is TempData in ASP.NET Core MVC?

TempData is a dictionary object provided by ASP.NET Core MVC that stores data temporarily until it is read. It allows data to persist from one HTTP request to the next, making it ideal for passing data during **redirects**.

- It stores data in key-value pairs, where the key is a string and the value is of object type. This allows us to store any type of data, including null values.
- Data in TempData persists only until it is read or until a subsequent HTTP request is made. Once read, it is automatically removed.
- It is specifically useful when you need to transfer small amounts of data between actions or views, for example, after form submissions with redirects.

```
public ITempDataDictionary TempData { get; set; }
```

Why Do We Need TempData in ASP.NET Core MVC?

There are scenarios in web applications where you need to pass data from one action to another, especially after a redirect. For example:

- After submitting a form (POST), you want to display a confirmation message on a GET action (PRG Pattern: Post-Redirect-Get).
- Passing error/success messages between controllers or actions after some processing.
- Avoid passing data in query strings or hidden form fields for short-lived data.
- Temporary status data or flags that should be available only for one subsequent request.

ViewBag and ViewData cannot be used for this, because they don’t persist across redirects. TempData fills this gap by retaining its data until it is read in the subsequent request.

TempData Methods: Load, Save, Keep, and Peek

```
public interface ITempDataDictionary : IDictionary<string, object?>
{
    void Keep();
    void Keep(string key);
    void Load();
    object? Peek(string key);
    void Save();
}
```

TempData Methods: Load, Save, Keep, and Peek

Load and Save Methods

These methods are rarely used directly in normal controller code. They are used internally by the framework:

- **Load**: Loads TempData from the session or cookies at the start of a request.
- **Save**: Saves TempData to the session or cookies at the end of a request, if there's still data remaining.

Under the hood, TempData uses either cookies or the session to store its data, depending on your configuration. By default, it uses cookies.

TempData Methods: Load, Save, Keep, and Peek

Keep() and Keep(string key)

By default, the TempData in ASP.NET Core MVC is designed to remove data after it is read, but sometimes you may want to keep data for more than one request. That's where Keep() and Keep(string key) come into play:

- **Keep()**: Marks **all** items in TempData for retention for another request, so nothing gets removed.
- **Keep(string key)**: Marks a **specific** key to retain its value for another request, instead of everything.

TempData Methods: Load, Save, Keep, and Peek

Peek(string key)

Allows you to read the value of a specific TempData key **without marking it for deletion**. In other words, you get the value, but it is still available for the next request.

How Do You Pass and Retrieve Data from TempData?

The most important point to remember is that it stores the data as an object, so when retrieving the data from TempData, type casting is required. If you are accessing string values from TempData, type casting is not necessary. However, it is mandatory to typecast explicitly to the actual type if you are accessing data other than the string type from the TempData.

Passing Data to TempData:

In your controller action, you can assign values to TempData using keys:

```
public IActionResult SubmitForm()
{
    // Set a TempData entry
    TempData["Message"] = "Your form was submitted successfully!";
    return RedirectToAction("Success");
}
```

How Do You Pass and Retrieve Data from TempData?

Retrieving Data from TempData:

In the action or view that receives the redirect, you can access the TempData value:

```
public IActionResult Success()
{
    // Retrieve and consume TempData
    var message = TempData["Message"] as string;
    return View(model: message);
}
```

In the Razor view, you can access it like:

```
@if (TempData["Message"] != null)
{
    <div class="alert alert-success">@TempData["Message"]</div>
}
```

How Do You Pass and Retrieve Data from TempData?

Retaining Data for Subsequent Requests:

To retain the value for subsequent requests, use the Peek or Keep method:

```
public IActionResult Success()
{
    // Approach 1: Read without removing
    var message = TempData.Peek("Message") as string;

    // Approach 2: Keep the key-value for another request
    TempData.Keep("Message");

    return View();
}
```

How TempData Manages the Data in MVC:

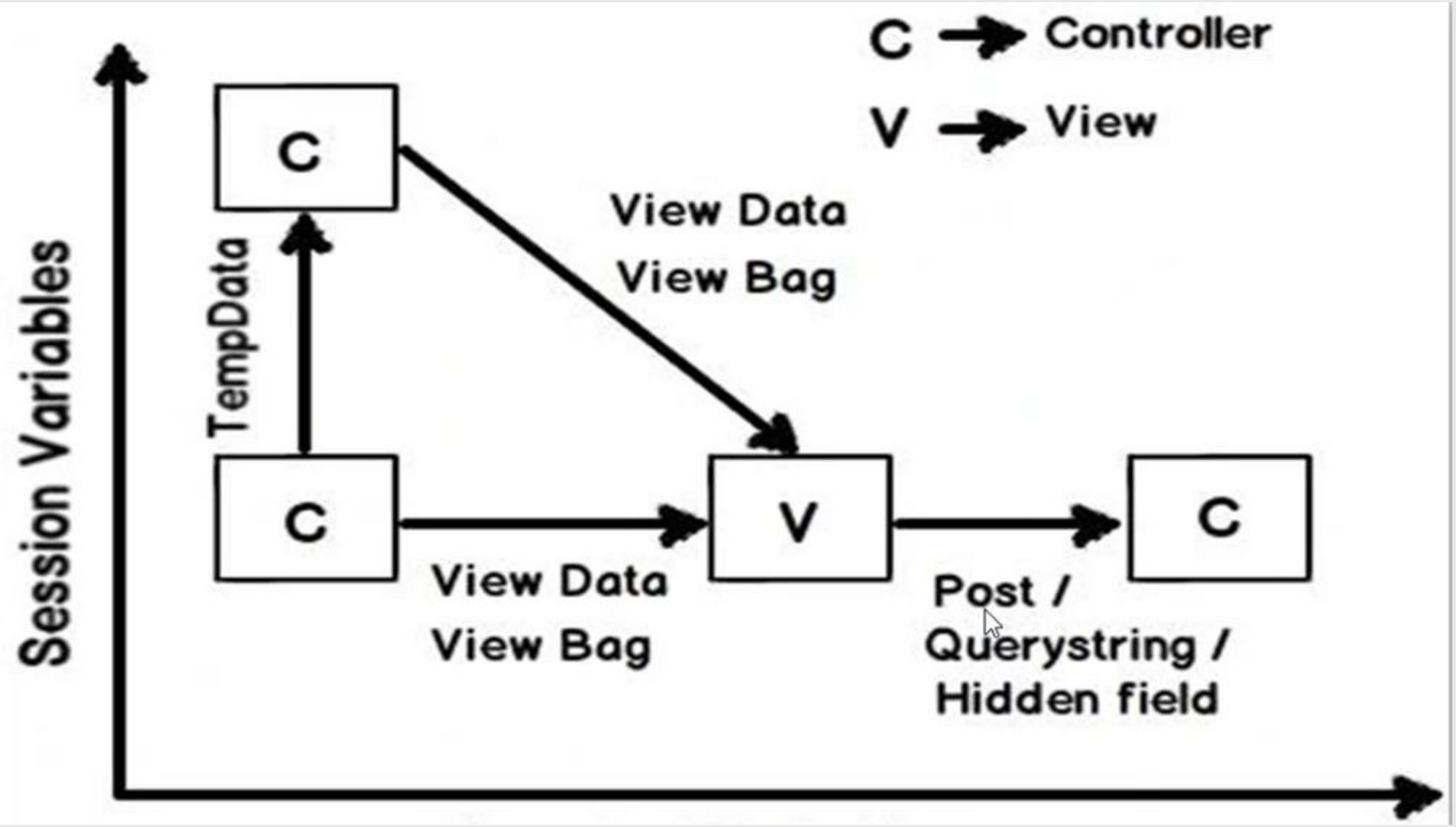
ASP.NET Core MVC provides different storage mechanisms for TempData, known as TempData providers. Depending on the configuration and application requirements, TempData can be stored using cookies or a session state.

- **Cookie-Based TempData Provider:** This provider stores TempData using cookies. The data is serialized into JSON, encoded, and stored as cookies. This method is stateless but limited by the size of cookies and involves exchanging data between the server and the client.

- **Session-Based TempData Provider:** This provider stores TempData in a session state. It’s server-side and can handle larger amounts of data compared to cookies. However, it requires the application to enable the session state.

Note: In ASP.NET Core MVC, if we don’t configure any Provider, it will use the Cookie TempData Provider by default.

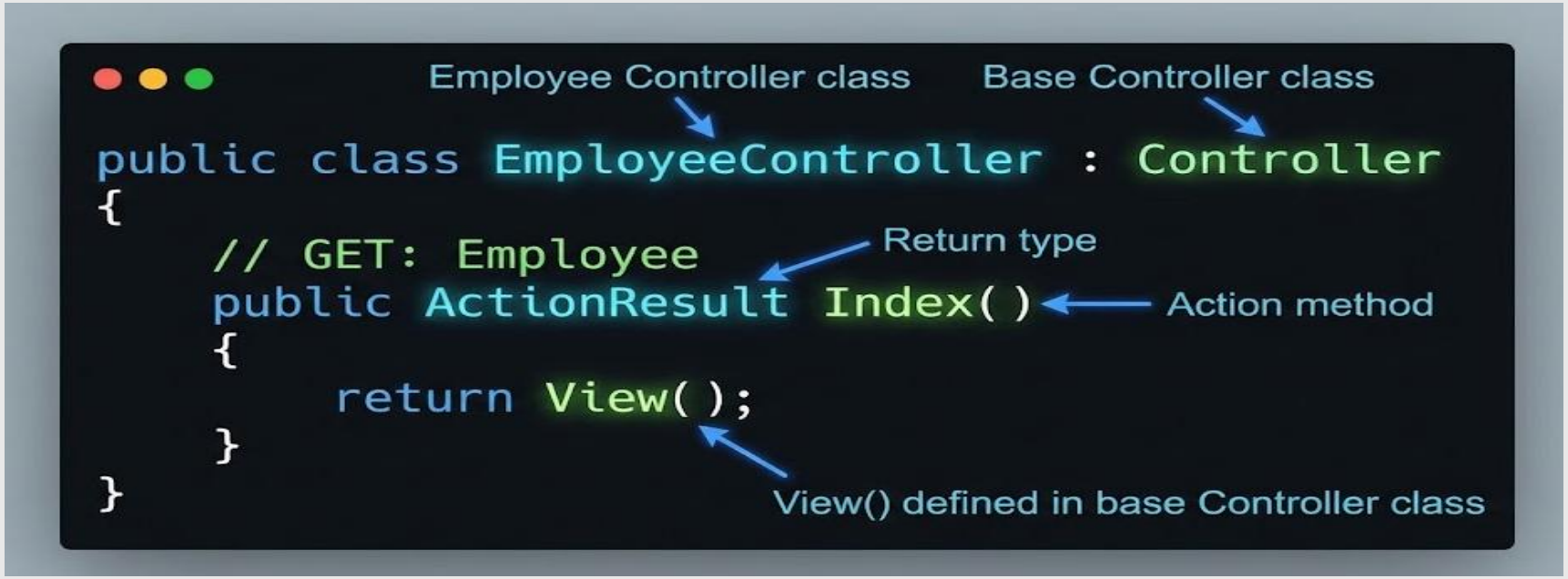
Session Variables



Action Method

Action method

- Public methods of a Controller class
- Action method must be public. It cannot be private or protected.
- Action method cannot be overloaded.
- Action method cannot be a static method.



Action method Parameters

- Every action methods can have input parameters as normal methods. It can be primitive data type or complex type parameters.
- Action method can include Nullable type parameters.

Parameter Sources

- Route Values: GET /users/5
- Query String: GET /users/search?name=Ali&age=25
- Request Body: POST /users { "name": "Ali", "age": 25 }
- Headers: Authorization: Bearer token123

Action method Parameters

- When a request hits a controller action, ASP.NET Core looks at different parts of the HTTP request to fill the method parameters.

How ASP.NET Core retrieve request data and map it to action parameters?

Without Model Binding

How to capture request data in an ASP.NET Core MVC Application without Model Binding. In ASP.NET Core MVC, we can use the IFormCollection interface to extract data manually from HTTP requests.

What is IFormCollection in ASP.NET Core MVC?

IFormCollection is an interface in ASP.NET Core MVC that represents a collection of form data submitted in an HTTP POST and PUT requests. It provides access to form fields and their values, allowing developers to work with the form data directly. The form fields are stored as key-value pairs, where the key is the field name, and the value is the field value(s). It provides the following methods and properties to retrieve form values:

- **Keys:** Retrieves all the keys in the form collection. It will hold all the form field names.
- **ContainsKey(key):** Determines if a specific key is present in the form collection.
- **TryGetValue(key, out value):** Attempts to get the value associated with a specific key.

The IFormCollection can be used as a parameter in an action method to access form data directly without binding each form field to a model property.

What are the Limitations of IFormCollection in MVC?

The following are the limitations of IFormCollection in ASP.NET Core MVC:

- **Limited to Form Data Only:** IFormCollection can only handle form data submitted via POST requests. It cannot be used to access data from other sources, such as query strings, route data, or JSON bodies.
- **No Strong Typing:** The data in IFormCollection is accessed via key-value pairs, meaning you lose the benefits of strong typing. You need to manually cast or convert the data to the desired types, which can lead to errors if the conversion fails.
- **Lack of Validation:** IFormCollection does not automatically validate the data. You must manually check for required fields, data types, and other validation rules, increasing the risk of errors.
- **Scalability Issues:** For large and complex forms, manually extracting and processing each form field using IFormCollection can become difficult, time-consuming, and less maintainable. As the number of fields increases, the code becomes more error-prone and harder to manage.
- **Manual Data Handling:** With IFormCollection, we need to handle data conversions, null checks, and errors manually. Model binding handles this automatically, making our code cleaner and easier to maintain.
- **No Built-in Security Checks:** IFormCollection does not inherently provide security features like anti-forgery token validation, which are more easily integrated when using model binding with view models.

Before Model Binding

- The traditional way to get data from http Request

HttpGet
/Employee/Edit?id=1

```
[HttpGet]
public ActionResult Edit()
{
    var id = Request.QueryString["id"];

    // retrieve data from the database

    return View();
}
```

HttpPost
/Employee/Edit

```
[HttpPost]
public ActionResult Edit()
{
    var id = Request["EmployeeId"];
    var name = Request["EmployeeName"];
    var age = Request["EmployeeAge"];

    // update database here..

    return RedirectToAction("Index");
}
```

Model Binding in ASP.NET Core

Microsoft Documentation

Dot Net Tutorials

What is Model Binding?

- Model Binding in ASP.NET Core MVC is the process by which HTTP request data (like form data, route data, query strings, and headers) is automatically mapped to action method parameters or model properties. The action method parameters can be simple types like integers, strings, etc., or complex types such as Student, Order, Product, etc.

- When a request is made to an MVC application, the data sent from the client is often in a format (such as query strings, form data, and route data) that needs to be converted into C# objects. Model Binding simplifies this process by handling this conversion automatically, allowing developers to work with strongly typed models instead of raw HTTP data.

What is Model Binding?

The HTTP Request can contain data in various formats. The data can contain in the HTML form fields. It could be part of the route values. It could be part of the query string or it may contain in the body of the request.

- Model Binding is a feature in ASP.NET Core
- It automatically maps HTTP request data to action method parameters if both names are matching
- Reduces manual parsing of request data
- Makes controllers clean and readable

How Does Model Binding Work in ASP.NET Core MVC?

Model binding in ASP.NET Core MVC works by inspecting the parameters of the action method and then looking through the sources of the request data (like form values, route data, query string parameters, etc.) to find matches. It uses various binders for different types of data. For example, simple types like int and string can be bound from route data or the query string, while complex types are typically bound from form values or JSON body data. The model binder is also responsible for validation based on data annotations or custom validators applied to the model properties. So, Model Binding in ASP.NET Core MVC works as follows:

- **Determine the Binding Source:** ASP.NET Core MVC first determines the source of the data based on the action method's parameters. Depending on the source attribute (e.g., [FromQuery], [FromBody], [FromRoute]), it knows where to look, such as in the query string, the request body, or the route data.

How Does Model Binding Work in ASP.NET Core MVC?

- **Fetch Data from the Source:** The framework retrieves the data from the identified sources.
- **Match and Convert Data:** The model binder matches the data with the action method parameters or model properties. If the names match, the data is converted to the appropriate type.
- **Bind Data to Parameters or Models:** The converted data is then bound to the parameters or properties.
- **Handle Errors:** If binding fails (e.g., due to a type mismatch), the model binder may add errors to the ModelState dictionary, which can be used to handle validation errors in the application.

Model Binding in ASP.NET Core

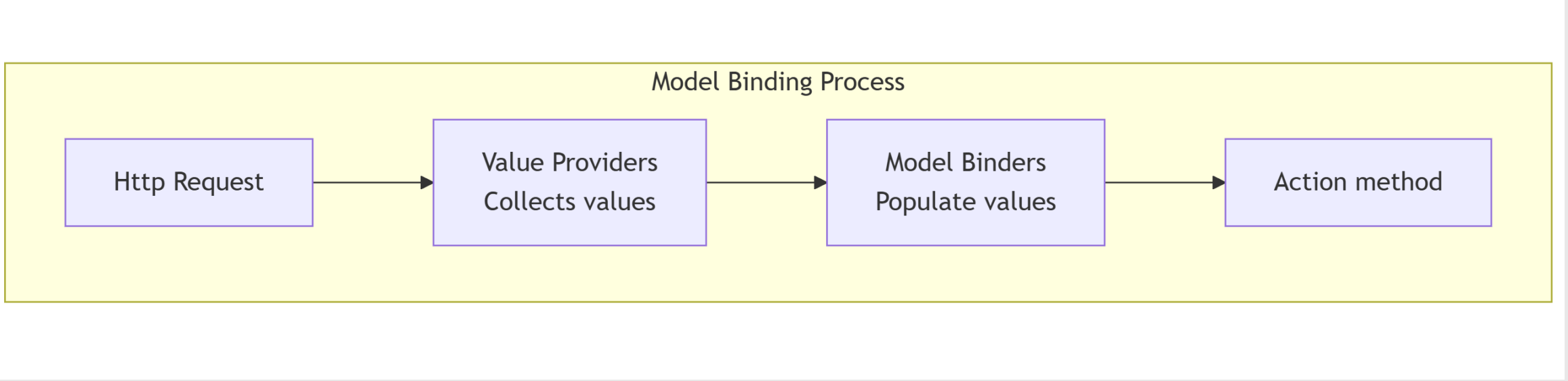
ASP.NET Core can bind data from:

- Route values
- Query string
- Form data
- Body (JSON)
- Headers

Model Binder

Model Binding is Two step Process:

- Collecting values from request using **Value Providers**
- Populating models with those values using **Model Binders**



Built in ValueProvider

Model Binders looks into Value Providers to find specific value on model property

Source	**Provider**	**Fetch Data From**
Form Fields	FormValueProvider	Request.Form
Route Data	RouteDataValueProvider	RouteData.Values
Query String	QueryStringValueProvider	Request.QuryString
Posted Files "FileUpload"	FormFileValueProvider	Request.Files

Default Model Binding

- When we define action method parameters, the values are auto populated in those parameters by the default model binder.

- There are two main points to consider in the Model Binding process
  - The name of the field defined in the request should be the same as the name of the action method parameter
  - The request data should be convertible to the action method parameter

Default Model Binding

- DefaultModelBinder class Maps a browser request to a data types of objects

- **Primitive Type Like:**
  - String, double, or DataTime objects
- **Collection Like**
  - Icollection<T>, Ilist<T>, Idictionary<TKey,TValue>
- **Model Class Like:**
  - Employee, Department …

Step 1: Collecting Values (Using Value Providers)

- Value Providers are responsible for reading raw data from the HTTP request.
- ASP.NET provides several built in value providers and also allows you to create your own.

Step 1: Collecting Values (Using Value Providers)

```
POST /users/5?active=true
Content-Type: application/json

{
  "name": "Ali",
  "age": 25
}
```

Value Providers collect:

- id = 5 (Route)
- active = true (Query)
- name = "Ali" (Body)
- age = 25 (Body)

At this stage:

- No C# objects yet
- Just key–value pairs

Step 2: Populating Models (Using Model Binders)

- Model Binders take the values collected in step 1 and convert them into C# parameters or objects.

What Model Binders Do

- Match values to parameter names
- Convert types ("25" → int)
- Create complex objects
- Handle collections (List<T>)
- Apply validation rules

Binding Attributes

- You can configure model binding using various attributes that control how data is extracted from the request. Some common attributes include:

- **[FromBody]:** This attribute tells ASP.NET Core to bind data from the request body to the parameter of an action method. It's commonly used when complex types like objects or collections, typically in JSON or XML format, are expected in the body of a POST request. It is commonly used in Restful Services.

- **[FromForm]:** This attribute binds data from HTML form fields to action method parameters. It's suitable for handling form submissions via HTTP POST or PUT requests and is commonly used in ASP.NET Core MVC Applications.

- **[FromQuery]:** Binds data from the query string of the URL to action parameters. The **[FromQuery]** attribute binds data from the URL's query string to action method parameters. It's often used in HTTP GET requests where data is passed in the URL as Query String Parameters.

- **[FromRoute]:** This attribute binds data from route parameters to action method parameters. Route parameters are placeholders in the URL defined in your route templates.

- **[FromHeader]:** This attribute helps extract custom or standard headers, such as Authorization, User-Agent, etc. This is often used in Restful Services.

Advantages of Model Binding

It simplifies accessing the request information in the action method

- No need to manually read from Request.Query, Request.Form, or Request.Body
- Controllers stay clean and focused

```
// Without model binding ❌
var id = int.Parse(Request.Query["id"]);

// With model binding ✅
public IActionResult Get(int id)
```

Advantages of Model Binding

Improves Readability & Maintainability

- Action method parameters clearly describe expected input
- Code is easier to understand and maintain

```
public IActionResult Update(int id, UserDto user)
```

Advantages of Model Binding

Automatic Type Conversion

- Converts string values to C# types automatically
- Handles primitives, enums, dates, and complex objects

```
age=25  →  int Age
```

Advantages of Model Binding

Supports Complex Objects

- Can bind nested objects and collections
- Ideal for real-world data models

```
public class OrderDto
{
    public int Id { get; set; }
    public List<ItemDto> Items { get; set; }
}
```

Advantages of Model Binding

Integrates with Validation

- Works seamlessly with Data Annotations
- Automatically detects invalid input
- With [ApiController], returns 400 Bad Request automatically

```
public class UserDto
{
    [Required]
    public string Name { get; set; }
}
```

Advantages of Model Binding

Multiple Data Sources Support

- Route
- Query string
- BodyHeaders
- Form data

Better Testing

Extensible & Customizable

Built-in Model Binders in ASP.NET Core MVC

ASP.NET Core MVC includes several built-in model binders to handle different types of data. They are as follows:

- **Simple Type Model Binder:** Binds simple types like integers, strings, booleans, etc.
- **Complex Type Model Binder:** Binds complex types (e.g., classes) by recursively binding their properties.
- **Array and Collection Model Binder:** Binds arrays and collections like List<T>, IEnumerable<T>, etc.
- **Dictionary Model Binder:** Binds dictionary types like Dictionary<TKey, TValue>.
- **DateTime Model Binder:** Specifically handles binding DateTime types.
- **FormFile Model Binder:** Binds uploaded files using IFormFile.
- **FormCollection Model Binder:** Binds form data using IFormCollection.
- **Complex Object Model Binder:** Binds complex objects that may contain other complex types or collections.
- **Enum Model Binder:** Binds enum values.

Layout View in ASP.NET Core MVC

What is Layout View in ASP.NET Core MVC?

- In ASP.NET Core MVC, the Layout View is a shared template or master page that can be used across different views within your application to maintain consistent looks and feels. It is useful for defining elements that are common across multiple pages, such as headers, footers, navigation bars, and sidebars. This helps maintain a consistent look and feel across your web application and also minimizes code duplication.

- So, a Layout View in ASP.NET Core MVC contains HTML markup and Razor code that will be rendered in other views that use this layout. For example, you can define a layout with a common header, footer, and navigation menu and then specify this layout in your individual views. The Layout Views include the following sections:

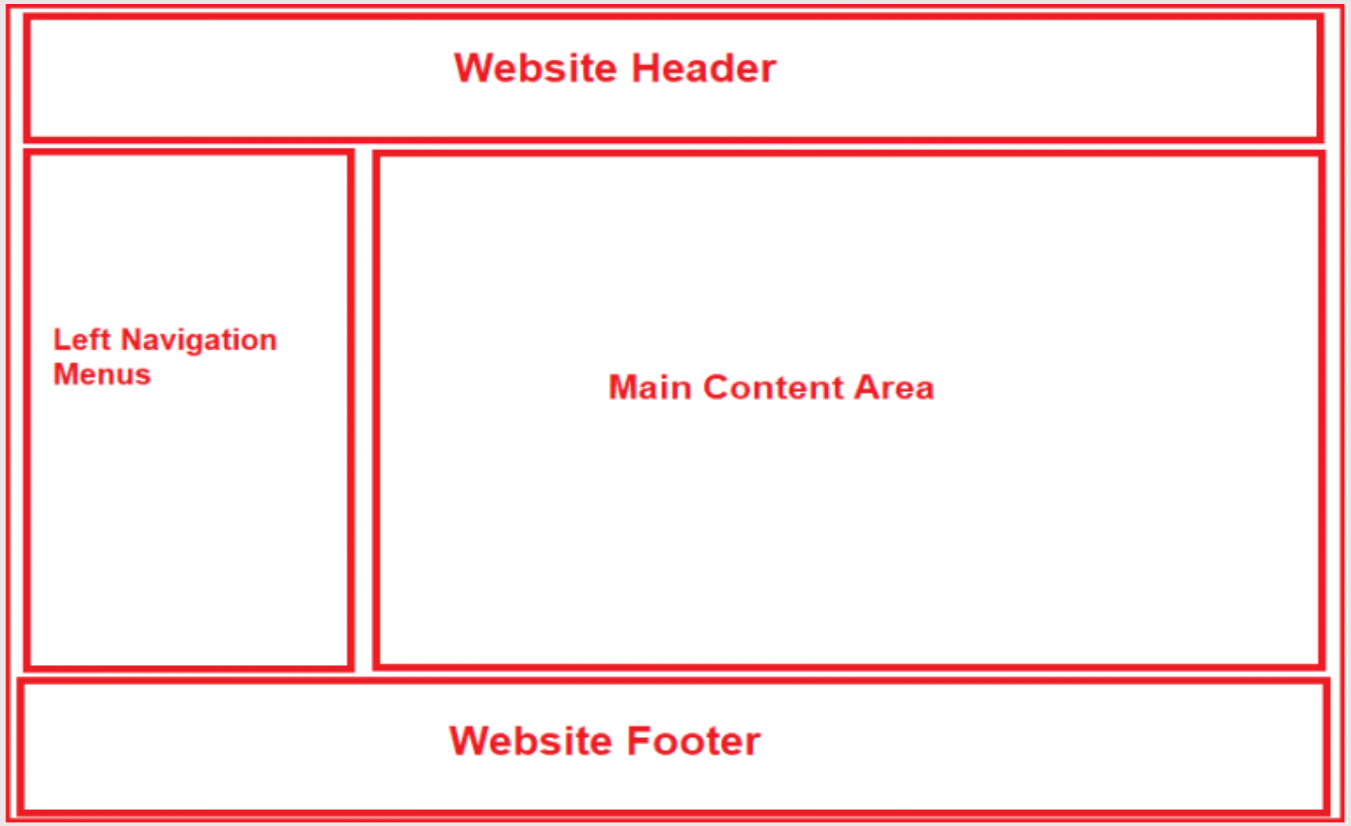
  - **@RenderBody():** This is a placeholder where the content of the views that use this layout will be rendered.
  - **@RenderSection("SectionName", required: false)**: This method is used to render optional sections that views might provide. If required is set to true, the section must be defined in the views.

Why Do We Need Layout View in ASP.NET Core MVC?

The layouts in ASP.NET Core MVC Applications help us maintain a consistent look across all the pages of our application. Nowadays, most web applications have a common layout that provides a consistent user experience when the user navigates from one page to another. The layout typically includes common user interface elements

such as:

- **Website Header**
- **Website Footer**
- **Navigation Menus**
- **Main Content Area**



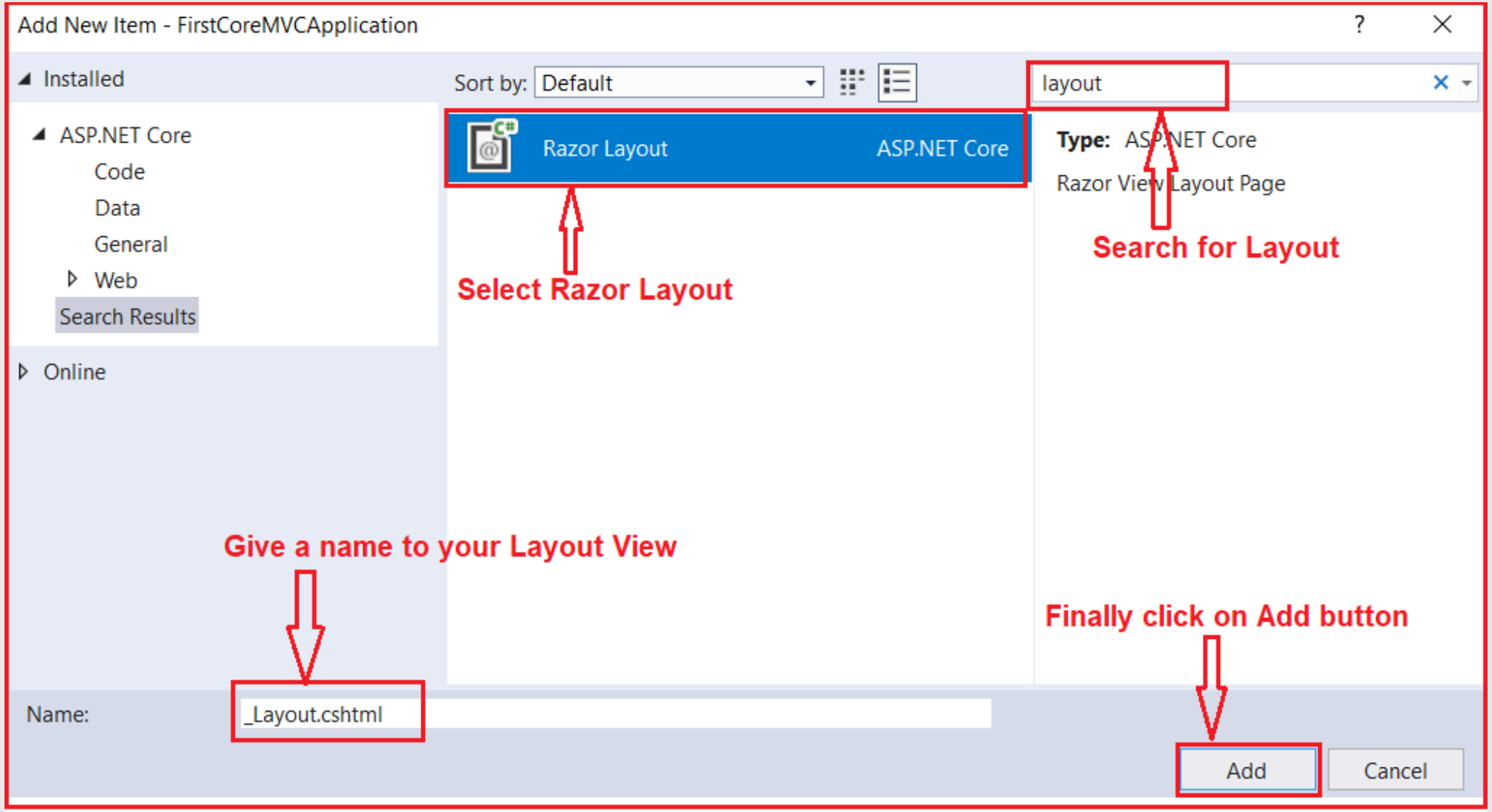
Why Do We Need Layout View in ASP.NET Core MVC?

- If you don’t have a layout view for your website, then you need to repeat the required HTML for the above-mentioned sections in each and every view of your application. This violates the **DRY (Don’t Repeat Yourself)** principle as we are repeating the same code in multiple views.

- As a result, it isn’t easy to maintain the application code. For example, suppose we have to remove or add a menu item from the list of navigation menus or even if we want to change the header or footer of our website. In that case, we need to do this in every view, which is tedious, time-consuming, and error-prone.

- Instead of putting all the sections (i.e., the HTML) in every view page, it is always better and advisable to put them in a layout view and then inherit that layout view in every view where we want that look and feel. With the help of layout views, it is now easier to maintain our application’s consistent look and feel. This is because if we need to make any changes, we need to do it only in one place, i.e., in the layout view, and the changes will be reflected immediately across all the views inherited from the layout view.

Create a Layout View in ASP.NET Core MVC Application



Structure of Layout View in ASP.NET Core MVC

```
<!DOCTYPE html>
<html>
<head>
    <!-- Common head elements like title, stylesheets, etc. -->
</head>
<body>
    <header>
        <!-- Common header content -->
    </header>
    <nav>
        <!-- Common navigation menu -->
    </nav>
    <div class="container">
        @RenderBody() <!-- Placeholder for view-specific content -->
    </div>
    <footer>
        <!-- Common footer content -->
    </footer>
</body>
</html>
```

Sections in Layout View in ASP.NET Core MVC

What are Sections in Layout View?

- The layout view is a shared template used to maintain a consistent look and feel across all the views in our application. In ASP.NET Core MVC, sections within a Layout View provide a way to inject content from a child view into specific places in the layout. This feature is useful when we want to include page-specific scripts, styles, or other elements in the layout without affecting our main content area.

- Sections are defined in the Layout View using the **RenderSection** or **RenderSectionAsync** method and can be marked as optional or required. In the child views, we need to specify the content for these sections using the **@section** directive.

Why Do We Need Sections in a Layout View?

- Suppose we have a custom Javascript file (e.g., CustomJavascript.js), and we want this file to be used in some specific views. Let’s assume we want this file in the Index view but not in the Details view of the Home Controller. In such scenarios, we can use the Layout Section in ASP.NET Core MVC Applications.

RenderSection and RenderSectionAsync

- To use Section in ASP.NET Core MVC Layout Views, we are provided with the RenderSection and RenderSectionAsync methods:

```
public HtmlString RenderSection(string name);
public HtmlString RenderSection(string name, bool required);
public Task<HtmlString> RenderSectionAsync(string name);
public Task<HtmlString> RenderSectionAsync(string name, bool required);
```

- **name:** The name of the section.

- **required:** A boolean value indicating whether the section is mandatory. If set to true, an exception is thrown if the section is not defined on the content page. If set to false, the layout will render without the section if it's not present on the content page.

Note: The first overloaded version takes only the name parameter; in that case, by default, it sets the required parameter value to true. So, if you use the first overloaded version, then it will be a required section.

RenderSection vs RenderSectionAsync

- **Synchronous vs. Asynchronous:**
  - RenderSection is synchronous and blocks the execution until the section content is rendered.
  - RenderSectionAsync is asynchronous and does not block the execution, allowing other asynchronous operations to run concurrently.

- **Use Case:**
  - Use RenderSection when the content for the section does not involve any asynchronous operations.
  - Use RenderSectionAsync when the section content involves asynchronous operations, such as network calls, database queries, or any other asynchronous tasks.

ViewStart in ASP.NET Core MVC Application

What is _ViewStart.cshtml File in MVC?

- In ASP.NET Core MVC, the _ViewStart.cshtml file is a special view that allows us to set common configuration settings for views in our application. This file is typically used to define settings that should apply to all views, such as specifying a layout, setting view-specific data, or configuring view behavior.

- The primary purpose of _ViewStart.cshtml is to contain code that we want to execute before rendering each view in the directory where the _ViewStart.cshtml file exists and its subdirectories. The _ViewStart.cshtml is executed before each view is rendered.

Why Do We Need _ViewStart.cshtml File?

- The primary use of _ViewStart.cshtml is to specify a default layout for all views, so we don't need to set it individually in every view. For example, we have used the Layout Property to associate a view with a layout view, as shown below. So, basically, within the Child view, we have added the following statement to use the layout view.
  Layout = "~/Views/Shared/_Layout.cshtml";

- Suppose we have 100 views in our application, and all the views want to use the same layout file. Then, we need to set the Layout Property, i.e., **Layout = "~/Views/Shared/_Layout.cshtml";** in all 100 views. This violates the **DRY (Don't Repeat Yourself)** Principle.

How Do We Select a Layout Conditionally in ViewStart?

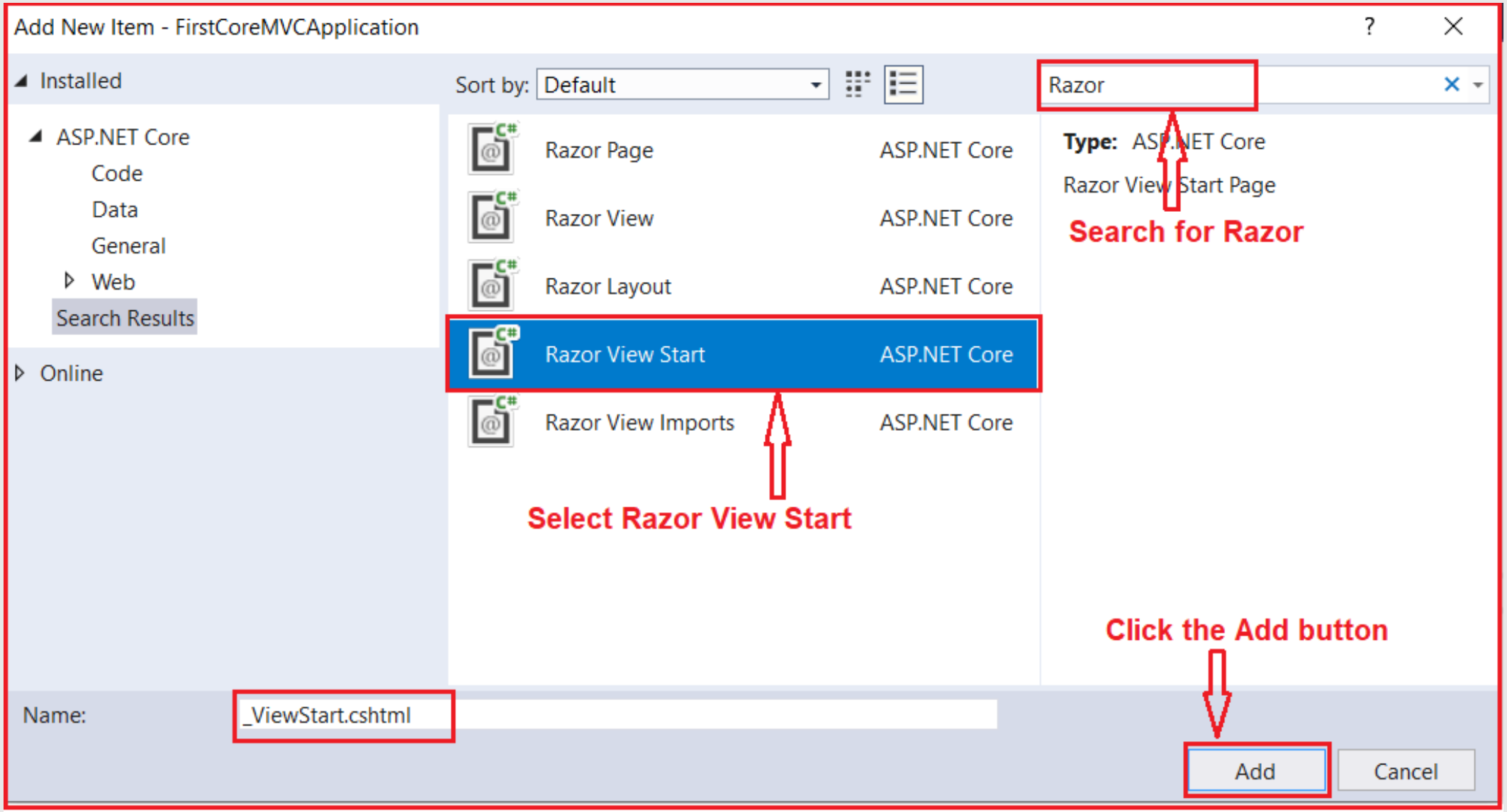
To conditionally select a layout in the **_ViewStart.cshtml** file in an ASP.NET Core MVC application, you can use C# code to set the layout based on certain conditions:

- **Identify the Condition:** First, you need to determine the condition for changing the layout. This could be based on the current controller, action, user roles, or any custom logic.

- **Implement the Conditional Logic:** Inside the _ViewStart.cshtml file, you can write C# code within @{ … } blocks. Using this C# code block, you can write your conditional logic to select the layout.

```
@{
    if (User.IsInRole("Admin"))
    {
        Layout = "_AdminLayout";
    }
    else
    {
        Layout = "_NonAdminLayout";
    }
}
```

How to Create _ViewStart.cshtml file in MVC?



ViewImports in ASP.NET Core MVC

What is _ViewImports.cshtml in MVC?

In ASP.NET Core MVC, the _ViewImports.cshtml file is a special view file that provides a way to include common C# directives and namespaces used across multiple views within an application. It helps to keep view files cleaner and more maintainable by allowing to define common settings such as namespaces, services, view components, and tag helpers in a single location. These settings are then automatically applied to all Razor views in the same directory or in any subdirectory.

Common Uses of _ViewImports.cshtml

- **Importing Namespaces:**
  - We can use _ViewImports.cshtml to import namespaces that are used across our Razor views. This can simplify our view code by allowing us to use types from those namespaces without having to specify @using directives in every view.
    @using MyApp.Models
    @using MyApp.ViewModels

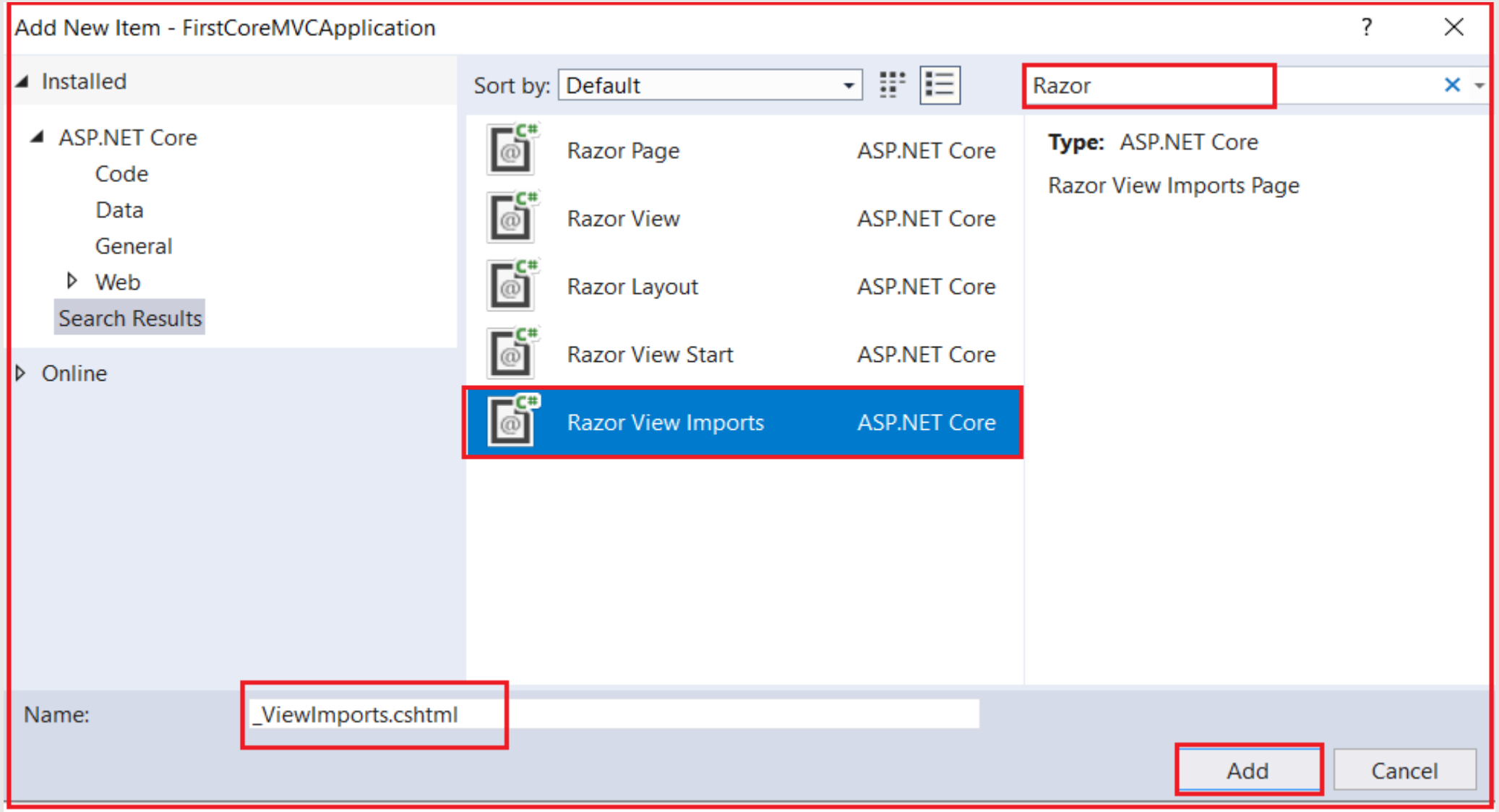
- **Tag Helpers:**
  - We can register tag helpers that we want to use in our Razor views. Tag helpers enhance the HTML markup in our views with server-side processing, eliminating the need to declare tag helpers in every view where they are used.
    @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

Common Uses of _ViewImports.cshtml

- **View Components:**
  - If we have custom view components that we want to use in your Razor views, we can also include them here.
    @using MyApp.ViewComponents

- **Injecting Services:**
  - We can also use the @inject directive in the _ViewImports.cshtml file to set up dependency injection for commonly used services across all Razor views. This feature allows us to inject services directly into the view, which can be very useful for accessing application data and functionality directly from within your views.
    @inject IHttpContextAccessor HttpContextAccessor

Creating ViewImports.cshtml file in MVC



Partial Views in ASP.NET Core MVC

What are Partial Views in ASP.NET Core MVC?

- Partial view is a reusable view, which can be used as child view in multiple other views.

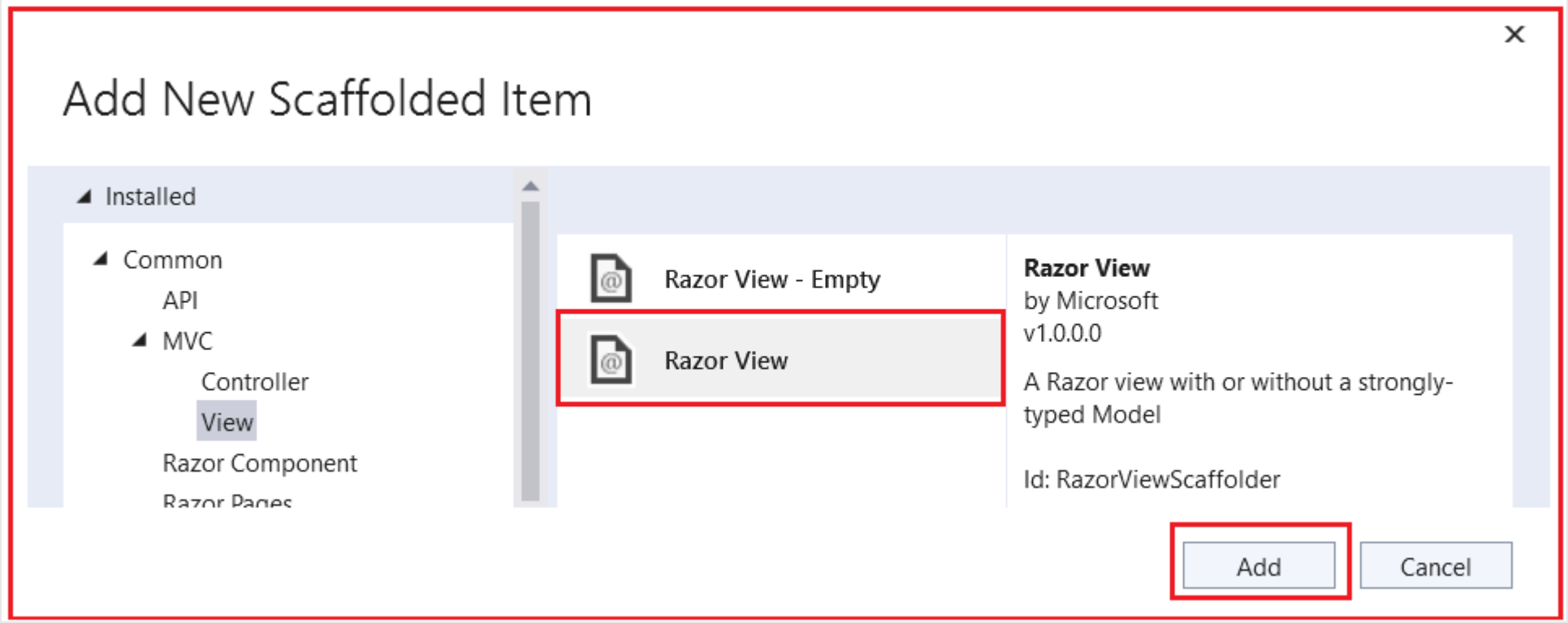
- We can use the partial view in the layout view, as well as other content.

- Partial Views are special views in the ASP.NET Core MVC Application that render a portion of the view content. They are used to encapsulate and reuse view logic and markup. They help break down large views into smaller, more manageable pieces, promoting reusability and maintainability by avoiding duplication of code. For example, you might have a partial view for a login form, a navigation menu, or a product summary that can be included in various views.

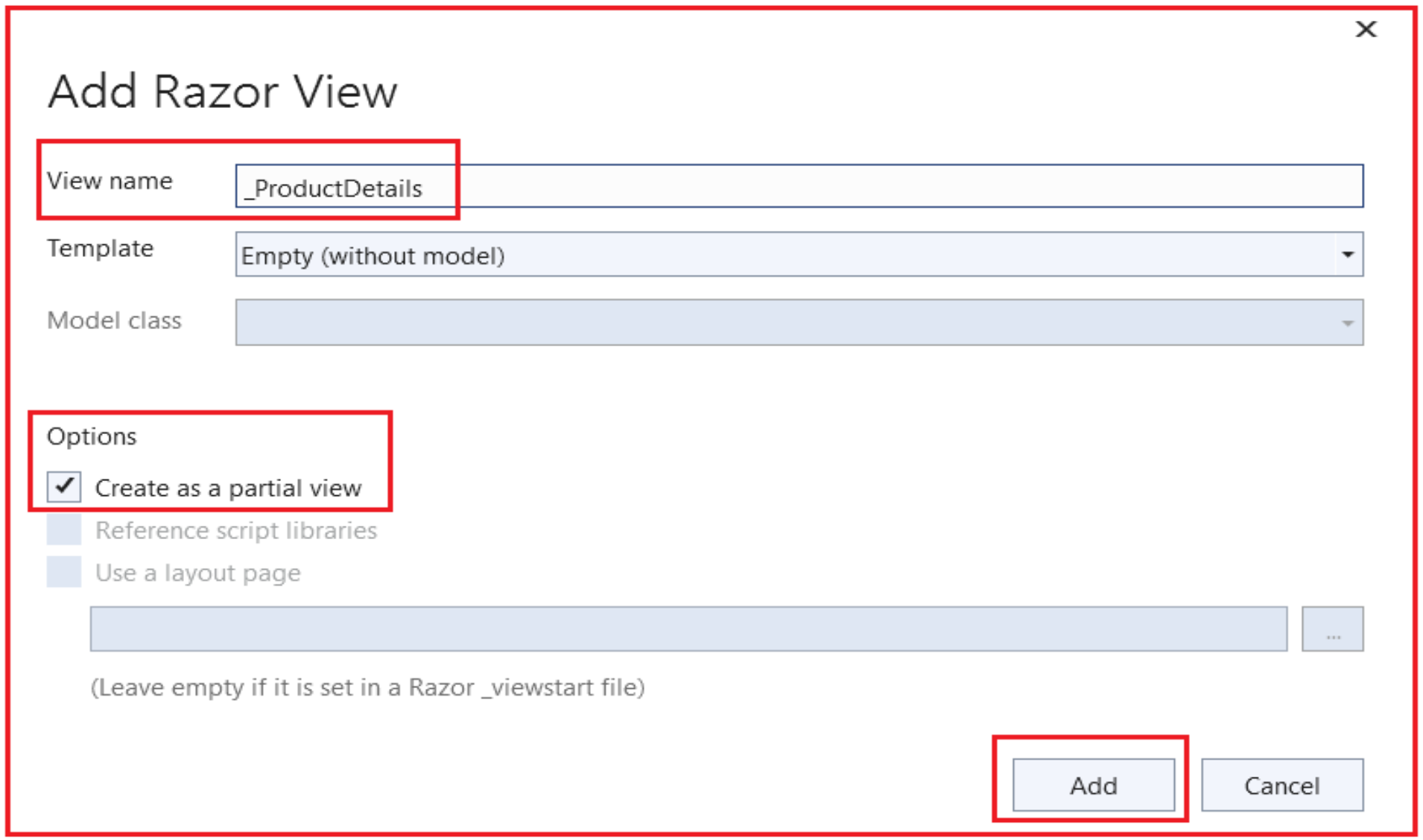
- Partial Views are similar to regular views, but they do not run on their own; they must be embedded within other views. This makes them ideal for rendering portions of HTML that are reusable across different parts of an application, such as headers, footers, or complex form elements. For example, if you have a section that displays user information that is required on multiple pages, you can create a Partial View for that section and include it wherever needed.

Create a Partial View in MVC

- Partial views are similar to regular views, but they start with an underscore **(_)** in their file name to indicate that they are meant to be partial.



Create a Partial View in MVC



How to Use the Partial Views in MVC

Method 1: Using Html.Partial

This is one of the simplest ways to render a partial view. It returns an HTML-encoded string and can be included directly within a Razor view. This method is executed synchronously. It is not recommended for use in modern applications because it blocks the calling thread until the view is rendered.

Syntax:

- **@Html.Partial("_PartialViewName")**
- **@Html.Partial("_PartialViewName", model)**

Arguments:

- The first argument is the name of the partial view.
- The second argument (optional) is the model passed to the partial view.

How to Use the Partial Views in MVC

Method 2: Using Html.RenderPartial

This method is similar to Html.Partial, but it renders the partial view directly to the response stream. This can improve performance because it doesn't require converting the partial view to a string before rendering. This method is also executed synchronously but does not return anything; it writes output directly. This method is useful when the partial view is large, and you want to avoid the overhead of string manipulation.

Syntax:

- **@{ Html.RenderPartial("_PartialViewName"); }**
- **@{ Html.RenderPartial("_PartialViewName", model); }**

How to Use the Partial Views in MVC

Method 3: Using Html.PartialAsync
This method is the asynchronous counterpart of Html.Partial. This method is used to asynchronously render partial views. It is the recommended way due to its non-blocking nature.

Syntax:

- **@await Html.PartialAsync("_PartialViewName")**
- **@await Html.PartialAsync("_PartialViewName", model)**

How to Use the Partial Views in MVC

Method 4: Using Html.RenderPartialAsync

This method is the asynchronous counterpart of Html.RenderPartial. This method is similar to Html.PartialAsync, but it directly writes the output to the response stream and does not return a result that can be captured.

Syntax:

- **@{ await Html.RenderPartialAsync("_PartialViewName"); }**
- **@{ await Html.RenderPartialAsync("_PartialViewName", model); }**

How to Use the Partial Views in MVC

Method 5: Using Tag Helpers

ASP.NET Core MVC provides a Partial Tag Helper that can render partial views. This is also executed synchronously.

Syntax:

- **<partial name="_MyPartialView" model="Model.PartialData" />**

Benefits of Using Partial Views in MVC

Modularity:

- Breaking down views into partial views makes our application modular and easier to manage.

DRY Principle:

- Partial views help adhere to the DRY (Don’t Repeat Yourself) principle by reusing common view logic. When we have a component (like a banner, a navigation menu, or a footer) reused across multiple pages.

Performance:

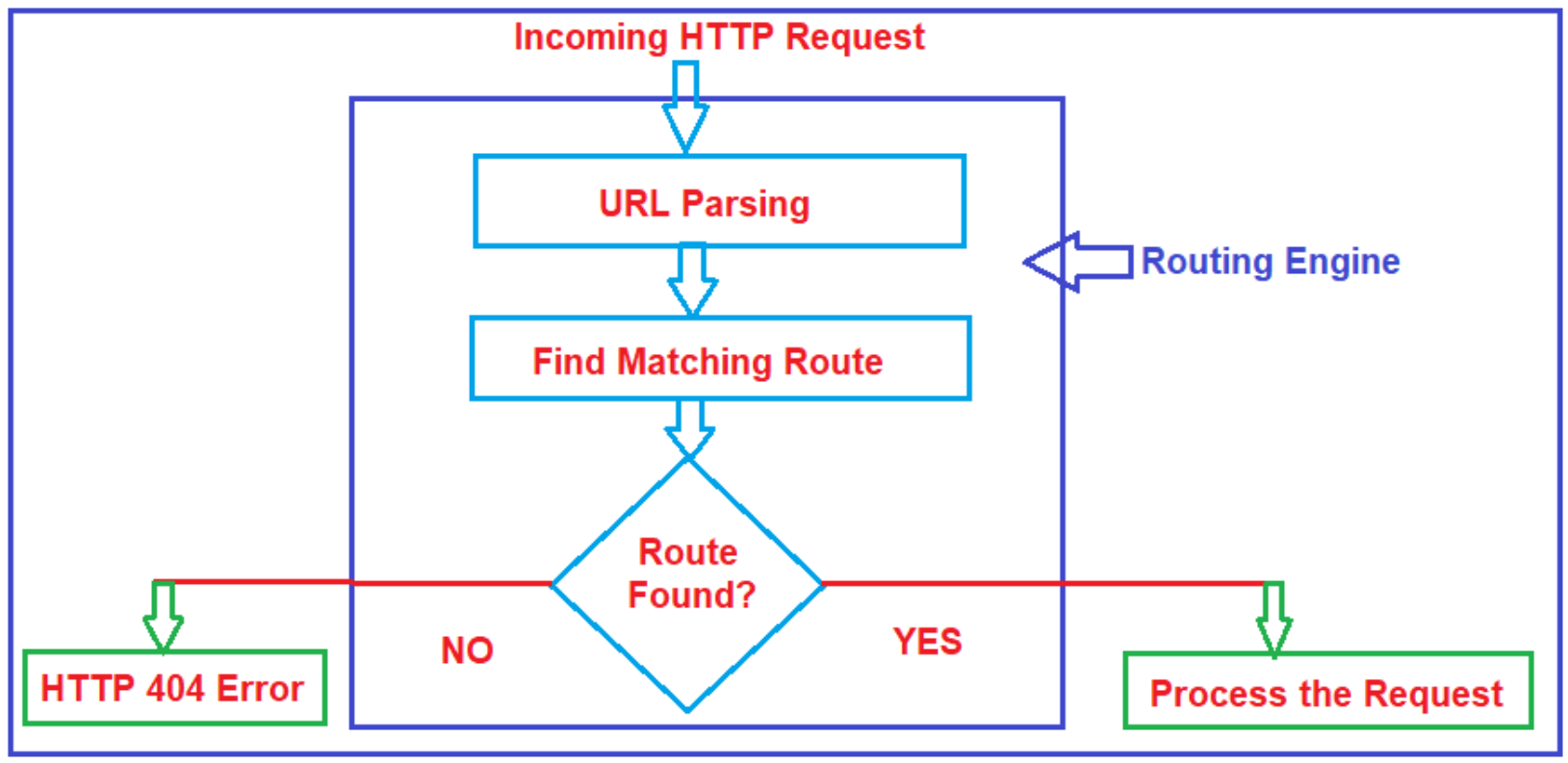
- Rendering only parts of a view can improve performance, especially when combined with AJAX requests to load partial content dynamically. Partial views can be returned from controller actions for AJAX calls to update a portion of a page without reloading the entire page.

Routing in ASP.NET Core MVC Application

What is Routing in ASP.NET Core MVC?

Routing in ASP.NET Core MVC is a mechanism that inspects the incoming HTTP request (i.e., URLs) and then maps those HTTP requests to the appropriate controller actions.

It enables the framework to determine what code to execute based on the URL of the request. Routing is an important component of ASP.NET Core MVC applications, providing the ability to define URL patterns and manage how URLs are processed and handled by the application.



Types of Routing Supported by ASP.NET Core MVC?

1. Convention-Based Routing
2. Attribute-Based Routing.

1. Convention-Based Routing

http://localhost:52190/Home/Index

```
public class HomeController : Controller
{
    public ViewResult Index()
    {
        return View();
    }
}
```

http://localhost:52190/Home/Details/2

```
public class HomeController : Controller
{
    public ViewResult Details(int id)
    {
        return View();
    }
}
```

1. Convention-Based Routing

- Routing Middleware Components are added using the UseRouting and MapControllerRoute methods in Program.cs

```
app.UseRouting();
```

This is Required for Routing

```
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

This will define the Route Pattern along with the Default Route

- **{controller}:** Represents the name of the controller class.
- **{action}:** Represents the action method name within the controller.
- **{id?}:** Represents an optional route parameter called “id”.

2. Attribute Routing in ASP.NET Core MVC

- Attribute Routing gives us more control over the URIs by defining routes directly at action methods or controllers in our ASP.NET Core MVC Web Application. With Attribute Routing, you can now define routes directly on actions and controllers by decorating them with attributes like [Route], [HttpGet], [HttpPost], etc. This type of routing makes it easier to understand the routes by looking directly at the controllers and actions.

```
[Route("Student/All")]
public List<Student> GetAllStudents()
{
    return students;
}
```

Attribute Routing vs Conventional Routing

Aspect	Conventional Routing	Attribute Routing
Characteristics	• Routes are defined globally in a centralized place (Program.cs). • Uses predefined route templates and naming conventions to map URLs to controllers and actions.	• Routes are defined using attributes directly on controllers and action methods. • Allows explicit and highly customized route templates.
Advantages	• Easy to manage and understand due to centralized configuration. • Ideal for applications with consistent URL patterns. • Keeps controllers clean without many route attributes.	• Provides fine-grained control over URLs. • Supports complex routes, constraints, and custom patterns. • Improves readability by keeping route definitions close to the actions they apply to.
Disadvantages	• Limited flexibility for action-specific or complex routes. • Becomes harder to manage in applications with highly dynamic URL patterns.	• Routing configuration can become scattered across controllers. • Harder to get a full picture of all routes in one place.
When to Use	• Large applications with standardized and predictable URL structures. Centralized route management. • Simple CRUD-based applications.	• Fine-grained URL Control • Complex Route Patterns • Versioning APIs • Mixing Static and Dynamic Segments

Route Constraints in ASP.NET Core MVC

- Route constraints in ASP.NET Core MVC are rules that can be applied to Route Parameters to restrict whether the route should be selected for a given request based on the values of those parameters. That means Route Constraints are a way to restrict or filter the values that a route parameter can take. These constraints help ensure that incoming requests match the expected format or data type before a controller action processes them.

- So, Route Constraints ensure that a route will only be selected if the parameters in the URL meet certain conditions. This helps in:
  - Validating input directly within the route setup reduces the need for additional validation within controllers.
  - Ensuring that URLs map to appropriate endpoints based on parameter types, formats, or values.
  - Avoid unnecessary processing for requests that do not meet specific criteria for route parameters.

```
app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "CustomRoute",
    pattern: "{controller}/{action}/{id:int}");
```

Accepting only int value

Types of Route Constraints

Rule / Type	Description
int	Matches any integer value
bool	Matches a Boolean value (true / false)
datetime	Matches a valid DateTime value
decimal	Matches a decimal number
double	Matches a double-precision number
guid	Matches a valid GUID value
long	Matches a long integer value
min(value)	Matches an integer greater than or equal to the specified minimum value
max(value)	Matches an integer less than or equal to the specified maximum value
range(min, max)	Matches an integer within the specified range
alpha	Matches one or more alphabetical characters (A–Z, a–z)
regex(expression)	Matches input based on the specified regular expression
length(min, max)	Restricts the length of the parameter value

File Handling in ASP.Net Core

What is a File, and What Are the Different Types of Files?

- A file is a collection of data or information stored with a specific name and path on a storage medium (e.g., a hard disk, SSD, or cloud storage). In web applications, files commonly refer to images, documents (PDF, Word), videos, or any other binary/textual data a user may upload or download.

- Types of Files (common categories relevant to web applications):
  - Image files (JPEG, PNG, GIF, etc.).
  - Document files (PDF, DOCX, XLSX, etc.).
  - Audio/Video files (MP3, MP4, WAV, etc.).
  - Compressed/Archive files (ZIP, RAR, 7z, etc.).
  - Text/Configuration files (.txt, .xml, .json, etc.).

Why Do We Need File Handling in Web Applications?

In web applications, file handling is crucial for scenarios such as:

- **File Upload:** Users need to upload files such as profile pictures, product images, or documents.
- **File Download:** Users or administrators may need to download or retrieve these files for further use.
- **Data Management:** Files may be stored as part of user or product data in e-commerce, CRM, or other large-scale applications.

Different Ways to Implement File Handling in MVC

There are two ways to implement File Handling in ASP.NET Core MVC:

- Buffering
- Streaming

Buffering Approach to Implement File Handling in MVC

The entire file is read into memory by copying its content into a Memory Stream. Once the file is fully loaded into memory, we convert it into a byte array. Finally, we write the byte array to the disk. This is straightforward but not ideal for large files because it may consume too much memory.

```
// Assume 'file' is an IFormFile obtained from the HTTP request.
// 'uploadFolder' is the path where the file will be saved,
// and 'uniqueFileName' is a unique file name generated for the upload.
if (file.Length > 0)
{
    using var memoryStream = new MemoryStream();      // Create a new MemoryStream instance.
    await file.CopyToAsync(memoryStream);             // Copy the uploaded file into the memory stream.
    byte[] fileBytes = memoryStream.ToArray();        // Convert the stream into a byte array.

    // Determine the full file path.
    string filePath = Path.Combine(uploadFolder, uniqueFileName);

    // Write the buffered bytes to the file system.
    await System.IO.File.WriteAllBytesAsync(filePath, fileBytes);
}
```

Advantages vs Disadvantages of Buffering

- **Advantages of Buffering:**
  - It is simpler to implement.
  - Suitable for smaller files (e.g., images, PDFs that are not too large).
  - Straightforward validations (e.g., file size, extension) before saving.

- **Disadvantages of Buffering:**
  - Large files can consume significant memory, impacting server performance.

Streaming Approach to Implement File Handling in MVC

In the streaming approach, the file is copied directly from the request stream to a file stream without buffering the entire file in memory. Instead of loading the whole file into memory, we open a FileStream that writes directly to the disk. This approach is more memory-efficient for large files since it does not require buffering the whole file, which minimizes memory usage.

```
// Assume 'file' is an IFormFile obtained from the HTTP request.
// 'uploadFolder' is the path where the file will be saved,
// and 'uniqueFileName' is a unique file name generated for the upload.
if (file.Length > 0)
{
    // Determine the full file path.
    string filePath = Path.Combine(uploadFolder, uniqueFileName);

    using var fileStream = new FileStream(filePath, FileMode.Create, FileAccess.Write, FileShare.None);
    await file.CopyToAsync(fileStream);  // Directly copy the uploaded file's stream to the FileStream.
}
```

Streaming Approach to Implement File Handling in MVC

In the FileStream Constructor:

- **filePath** tells the system where to save the file.
- **FileMode.Create** ensures a new file is created (or an existing one is overwritten).
- **FileAccess.Write** allows data to be written to the file.
- **FileShare.None** ensures that no other process can access the file during this write operation.

Advantages vs Disadvantages of Streaming

- **Advantages of Streaming:**
  - It is ideal for large file uploads and reduces memory pressure on the server.
  - More scalable for large-scale or concurrent uploads.

- **Disadvantages of Streaming:**
  - More complex to implement.
  - Validations (like file size and extension) might be trickier when data arrives in chunks.

Which is Good from the Performance Point of View?

- Streaming is typically more performant and resource-friendly for large files because it prevents loading the entire file into memory. So, if your application needs to handle large files (hundreds of MBs or GBs) or you anticipate a large volume of concurrent uploads, go with Streaming.

- Buffering is fine for smaller files and much easier to implement and maintain. So, if your application deals with small to medium file sizes (such as images and standard PDFs), go with Buffering.

Restricting Uploaded File Type and Size in ASP.NET Core MVC

Why Do We Need to Restrict File Type and File Size in Web Applications?

- When allowing file uploads in web applications, it is crucial to impose restrictions on file type and file size for several reasons:

  - **Security Concerns:** Allowing unrestricted file type uploads may lead to security vulnerabilities, such as uploading malicious scripts (e.g., .exe, .bat, .php, etc.). By restricting file types, we minimize the risk of executing harmful code.

  - **Storage Management and Performance Optimization:** Unrestricted large file uploads can lead to excessive disk usage, affecting server performance. Large file uploads can degrade the application's performance, slowing down the system and increasing server load. Limiting the file size Limiting file size and format ensures faster upload and download times, leading to a better user experience.

  - **Data Integrity and Compatibility:** Some applications expect specific file formats. Restricting to known file types ensures that uploaded files can be processed and used effectively by the application. Informing users about allowed file types and sizes improves usability and prevents frustration when uploads fail later.

What are File Extensions and MIME Types?

File Extensions and MIME Types

A file extension represents the type of file, while a MIME type (Multipurpose Internet Mail Extensions) describes the file's content. The following are some of the common file extensions and their corresponding MIME types:

File Extensions

A file extension is the suffix at the end of a filename, such as:

- **.jpg –** Commonly used for JPEG images
- **.png –** Portable Network Graphics image
- **.gif –** Graphics Interchange Format image
- **.pdf –** Portable Document Format
- **.docx** – Microsoft Word document
- **.xls (.xlsx) –** Microsoft Excel Files
- **.txt –** Text Files

File Extensions and MIME Types

MIME Types

MIME (Multipurpose Internet Mail Extensions) types are standardized identifiers used on the Internet to indicate the nature and format of a file, such as:

- **image/jpeg** – for JPEG images
- **image/png** – for PNG images
- **image/gif** – for GIF images
- **application/pdf** – for PDF documents
- **application/msword** – for DOCX files
- **application/vnd.ms-excel** – for Excel Files
- **text/plain** – for Text Files

Note: It is good practice to validate both the file extension (to check the filename) and the MIME type (to check the actual content type).

File Storage Strategies in ASP.NET Core Applications

File Storage Strategies in ASP.Net Core

In web applications, files can be stored in different ways depending on:

- File size
- Performance requirements
- Security needs
- Scalability
- Budget

There is no single “best” solution — it depends on the system design.

File-saving options

Available Options for Saving Files:

- Store Files Inside Database (Binary Data)
- Store Files Inside Database (Base64 String)
- Store Files in Server File System
- Store Files on Separate File Server
- Cloud Storage (Most Scalable Solution)

Store Files Inside Database (Binary Data)

Store file content as VARBINARY(MAX) in Microsoft SQL Server.

How It Works:

- Convert file to byte[]
- Save inside database table

Advantages

- Centralized storage
- Easier backup
- Secure access control
- Good for small files

Disadvantages

- Large database size
- Slower performance with large files
- Higher cost

Store Files Inside Database (Base64 String)

Convert file to Base64 string and store as text in database.

Advantages

- Easy to transmit via JSON / APIs
- Useful in small systems

Disadvantages

- Increases file size (~33%)
- Inefficient for large files
- Not recommended for production

Store Files in Server File System

Store files in application folder (e.g., wwwroot/uploads) then Store only file path in database.

Advantages

- Better performance
- Smaller database
- Easy to implement
- Common practice

Disadvantages

Must manage file cleanup

Backup requires DB + file system

Store Files on Separate File Server

Instead of saving inside web server:

- Use dedicated file server (NAS / network storage)
- Save file path in database

Advantages

- Better scalability
- Separates storage from application server
- Handles larger file volumes

Disadvantages

- Requires infrastructure setup
- Network dependency

Cloud Storage (Most Scalable and Production Solution)

Store files in cloud storage services:

- Azure Blob Storage
- Amazon S3
- Google Cloud Storage
- Save only file URL in database.

Advantages

- Highly scalable
- Global access
- Cost efficient
- Built-in redundancy & backup

Disadvantages

- Requires internet access
- Ongoing subscription cost

Comparison Summary

Storage Method	Best For	Scalability	Cost
Database	Small files	Low	Medium
Server Folder	Medium files	Medium	Low
File Server	Large internal systems	High	Medium
Cloud Storage	Large systems	Very High	Pay-as-you-go
Hybrid	Enterprise systems	Very High	Optimized

Middleware in ASP.Net Core

Microsoft Documentation

Dot Net Tutorials

Middleware in ASP.Net Core

- A middleware is nothing but a component (class) which is executed on every request in an ASP.NET Core application.
- There will be multiple middleware in an ASP.NET Core web application. It can be either framework-provided middleware, added via NuGet, or your own custom middleware.
- Each middleware adds or modifies the HTTP request and optionally passes control to the next middleware component.

Middleware in ASP.Net Core

- The request handling pipeline is composed as a series of middleware components.
- Each Component in the pipeline is a request delegate.
- Request Flow: When a component stops the forward flow, it begins returning back up the call chain.

What are ASP.NET Core Middleware Components?

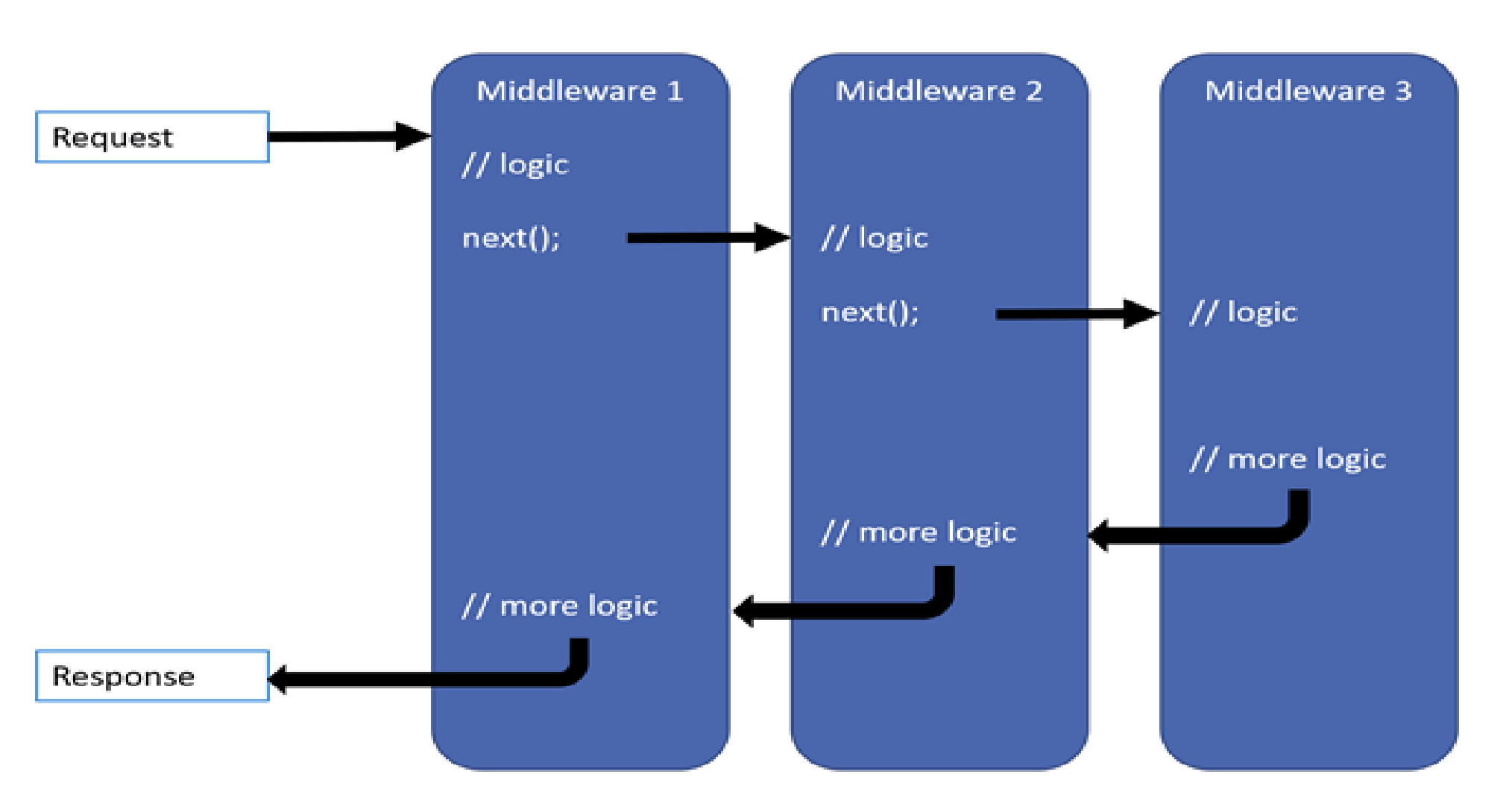
ASP.NET Core Middleware Components are the basic building blocks of the Request Processing Pipeline in ASP.NET Core Applications. The Request Processing Pipeline determines how HTTP Requests and Responses will be processed in the ASP.NET Core Application. So, middleware components are used to implement various functionalities like authentication, authorization, error handling, routing, and logging.

Middleware components are executed in the order they are added to the pipeline, and each middleware component in the ASP.NET Core Application performs the following tasks.

- Chooses whether to pass the HTTP Request to the next Middleware component registered in the request processing pipeline.
- Can perform certain tasks before and after the next component is invoked in the pipeline.

Note: In ASP.NET Core, many built-in Middleware components are already made available for us to use directly. We can also create our custom Middleware components as per our business requirements. The most important point you need to remember is that a given Middleware component should only have a specific purpose, i.e., a single responsibility.

Middleware in ASP.Net Core



Configure Middleware Components in ASP.NET Core

We need to configure the Middleware Components within the Main() method of the Program class, which is present inside the **Program.cs** class file using the WebApplication instance. When we create a new ASP.NET Core Empty Web Application, the Program class is created with the Main method by default, as shown in the image below. The following Main method configures two middleware components, **MapGet** and **Run, by default**.

```
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        var app = builder.Build();

        app.MapGet("/", () => "Hello World!");

        app.Run();
    }
}
```

Configure Middleware Components in ASP.NET Core

To configure Middleware Components, we can use the Map/MapGet, Run, and Use extension methods in ASP.NET Core applications.

- Execute Request then Call Next Middleware (Use)
- Execute Request then terminate (Run)
- Map (Map)

How Does the Middleware Component Works in the ASP.NET Core Application?

In the ASP.NET Core Web Application, the Middleware Component can access both the incoming **HTTP Request** and outgoing **HTTP Response**. So, a Middleware Component in ASP.NET Core Web Application can:

- Handle the incoming HTTP request by generating an HTTP response.
- Process the incoming HTTP request, modify it, and then pass it to the Next Middleware Component configured in the Request Processing Pipeline.
- Process the outgoing HTTP response, modify it, and then pass it on to either the previous middleware component or the ASP.NET Core Web Server.

What is the Execution Order of Middleware Components?

The execution order of Middleware Components in ASP.NET Core Web Applications. These components are executed in the same order as they are added to the Request Processing Pipeline for incoming requests and in reverse order for outgoing responses. So, we must take proper care when adding the Middleware Components to the Request Processing Pipeline. If we add them in the wrong order, then we might get unexpected behavior.

Types of Middleware in ASP.Net Core

Main **types of middleware in ASP.NET Core**:

- **Built-in Middleware:** ASP.NET Core provides a comprehensive set of built-in middleware
  - **Routing Middleware:** Matches incoming requests to endpoints.
  - **Endpoint Middleware:** Executes the matched endpoint.
  - **Static Files Middleware:** Serves static files (CSS, JS, images).
  - **Authentication Middleware:** Identifies the user.
  - **Authorization Middleware:** Checks user permissions.
  - **Exception Handling Middleware:** Handles errors globally.
  - **CORS Middleware:** Enables Cross-Origin Resource Sharing.
  - **HTTPS Redirection Middleware:** Redirects HTTP requests to HTTPS.

- **Custom Middleware:** Also ASP.NET Core allows developers the flexibility to write custom middleware.
  - **Class-based Middleware**
  - **Inline Middleware (Lambda-based)**

What is Custom Middleware?

In **ASP.NET Core**, custom middleware is code you write to:

- Intercept HTTP requests
- Execute logic before and/or after the next middleware
- Control the request pipeline

Real Uses of Custom Middleware

Logging Requests & Responses

Track:

- Request path
- HTTP method
- Response status code
- Execution time

Example use:

- Monitor API performance
- Debug production issues
- Store logs in database or file

Real Uses of Custom Middleware

Global Exception Handling

Instead of try/catch in every controller:

- Catch all unhandled exceptions
- Return unified JSON error response
- Log errors centrally

Very common in production APIs.

Real Uses of Custom Middleware

Authentication / Token Validation

Before request reaches controller:

- Check if JWT token exists
- Validate API key
- Block unauthorized users

Real Uses of Custom Middleware

Authentication / Token Validation

Before request reaches controller:

- Check if JWT token exists
- Validate API key
- Block unauthorized users

Real Uses of Custom Middleware

Request Validation

- Block specific IP addresses
- Restrict certain routes
- Check headers

Real Uses of Custom Middleware

Modify Request or Response

You can:

- Add custom headers
- Modify response body
- Standardize API response format

Real Uses of Custom Middleware

Performance Monitoring

Measure how long each request takes:

Useful for:

- Detecting slow endpoints
- Optimizing system

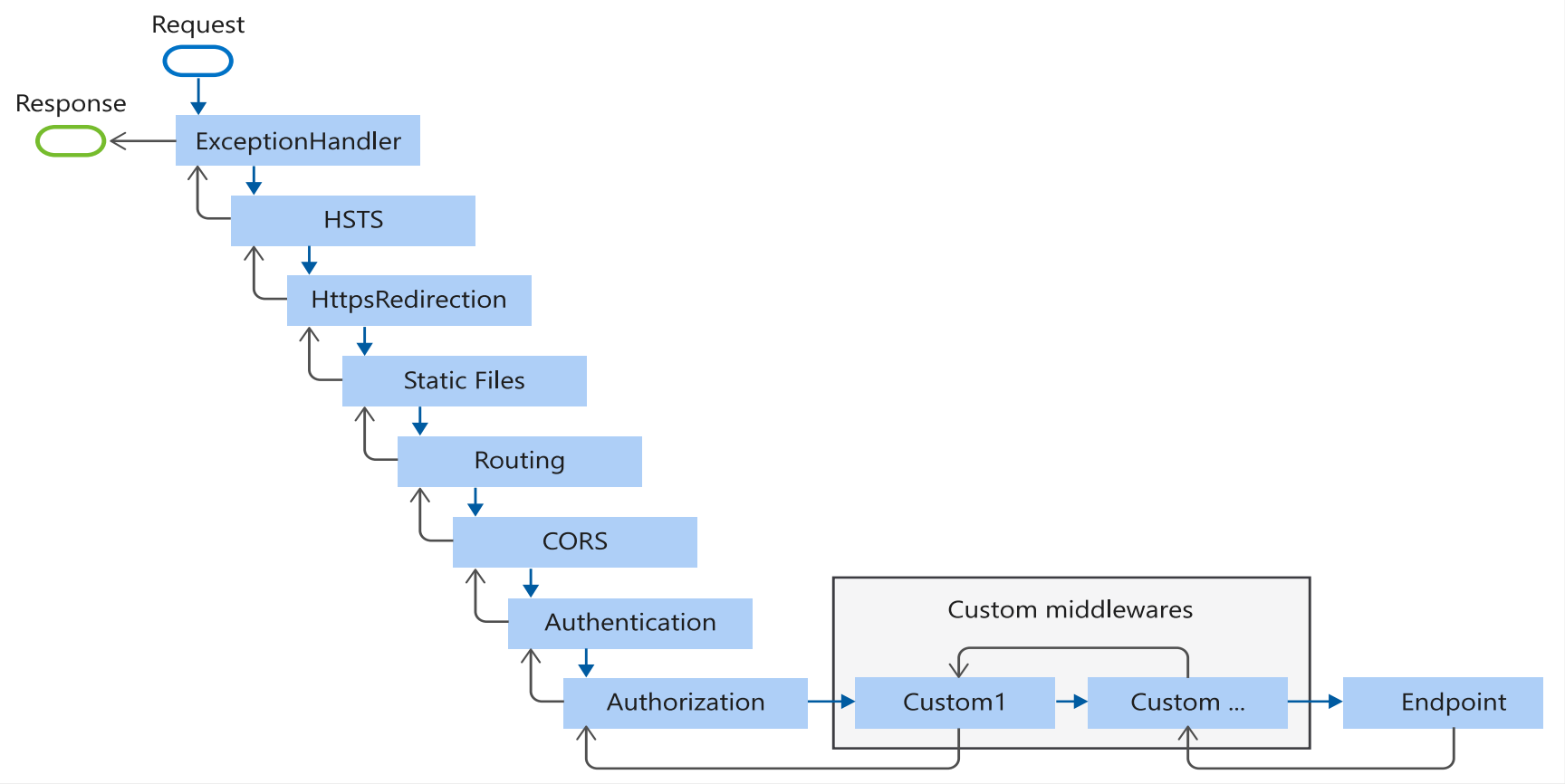
```
var watch = Stopwatch.StartNew();
await _next(context);
watch.Stop();
```

What is the ASP.NET Core Request Processing Pipeline?

The ASP.NET Core request processing pipeline is a series of middleware components that handle incoming HTTP requests and responses in an ASP.NET Core Web application. Each middleware component is responsible for a specific task, such as authentication, routing, logging, caching, encryption and decryption, response generation, etc. The pipeline is configured in the Program class of an ASP.NET Core application.

Request Processing Pipeline

This diagram shows the ASP.NET Core Request Processing Pipeline and the sequence in which middleware components should registered and executed when a request is processed in an ASP.NET Core application. Let us understand each step in the Request Processing Pipeline:



Request Processing Pipeline

- **Request:** This is where the incoming HTTP request enters the pipeline. Each middleware component in the pipeline can inspect, modify, or pass the request to the next component.

- **ExceptionHandler:** This middleware is often used to handle exceptions globally. If an unhandled exception occurs, it handles it, logs it, and returns a proper error response, usually a custom error page or a JSON error message.

- **HSTS (HTTP Strict Transport Security):** This middleware enforces strict security policies, telling browsers to only interact with the application over HTTPS. This means this middleware ensures that the application only serves content over HTTPS and prevents any HTTP requests by enforcing strict transport security policies. This middleware is often used in production environments to enhance security.

Request Processing Pipeline

- **HttpsRedirection:** This middleware redirects HTTP requests to HTTPS. By redirecting any HTTP request to HTTPS, this middleware ensures that all traffic is encrypted for security.

- **Static Files:** This middleware enables static files like HTML, CSS, JavaScript, and images to be served directly without passing them through the rest of the pipeline. This is useful for files that don’t need processing.

- **Routing:** This middleware maps the incoming request to a route, allowing the application to handle it appropriately. Based on URL patterns and route configurations defined in the application, this middleware determines which controller and action will handle the incoming HTTP request.

Request Processing Pipeline

- **CORS (Cross-Origin Resource Sharing):** This middleware handles CORS policies (cross-origin requests), allowing or denying requests from different origins. CORS is important for accessing resources from different domains and is essential for APIs used by frontend applications hosted on different origins.

- **Authentication:** This middleware handles the authentication or identity of the user or client making the request by validating credentials (e.g., JWT tokens, cookies, etc.). It ensures that the user is authenticated before reaching certain parts of the application.

- **Authorization:** This middleware determines whether an authenticated user is allowed to access a specific resource or endpoint. It checks if the authenticated user has permission to access the requested resource.

Request Processing Pipeline

- **Custom Middlewares (Custom1, Custom…):** This represents any custom middleware components you may have added to the pipeline. These custom middleware components can perform various operations on the request or response, such as logging, request modification, caching, modifying headers, custom validation, etc.

- **Endpoint:** This is the target of the request, such as a controller action in MVC, Web API, or a Razor Page. The endpoint processes the request and generates a response.

- **Response:** After the endpoint generates a response, it goes back through the pipeline, allowing each middleware component to modify it before sending it back to the client.

Filters in ASP.Net Core

Microsoft Documentation

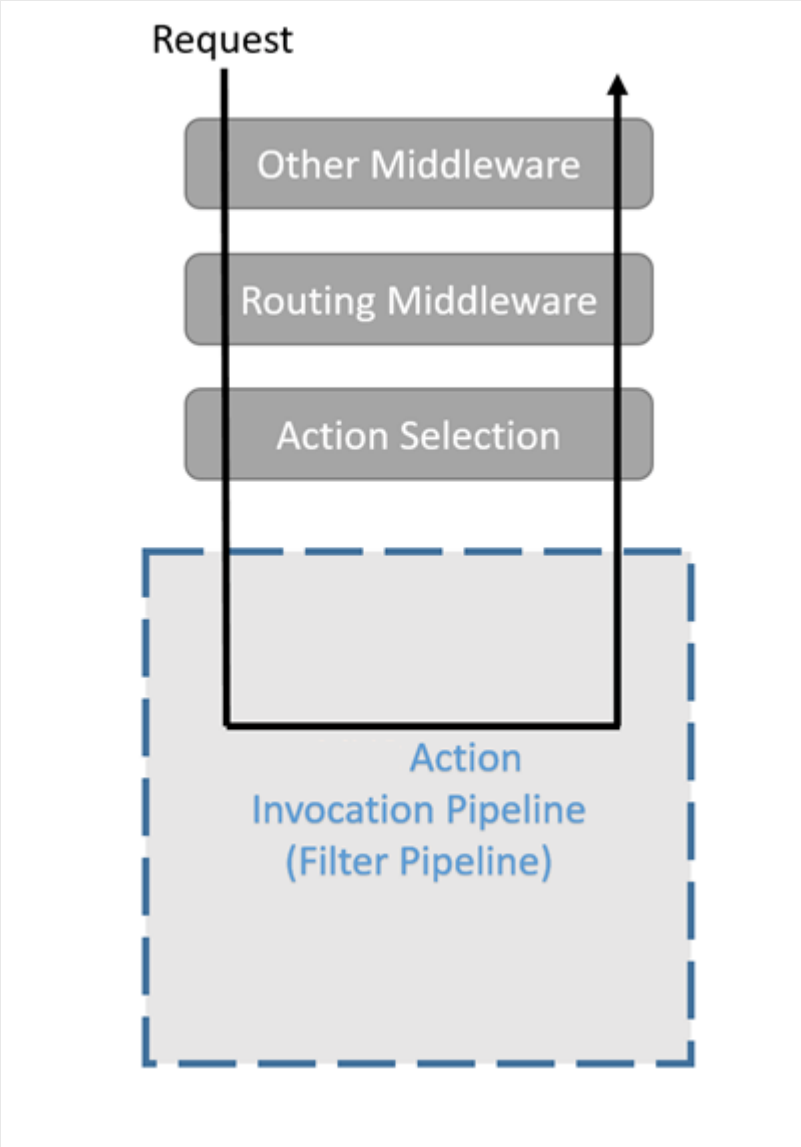
Dot Net Tutorials

What are Filters in ASP.NET Core?

- In ASP.NET Core, a filter is a piece of code that runs before or after specific stages in the request processing pipeline, allowing you to handle cross-cutting concerns like:
  - Authorization
  - Logging
  - Caching
  - Exception handling
  - Validation
  - Performance monitoring
- Custom filters can be created to handle specific concerns for your application.
- Filters also help to avoid code-duplication across action methods.

How filters work in ASP.NET Core?

Filters run within the ASP.NET Core action invocation pipeline, sometimes referred to as the filter pipeline. The filter pipeline runs after ASP.NET Core selects the action to execute:



Parent interface

Filters implement the IFilterMetadata interface in Microsoft.AspNetCore.Mvc.Filters namespace.

Filter Types in ASP.NET Core?

Filter Type	What It Does	Main Interfaces
Authorization Filters	• Run first. • Determine whether the user is authorized for the request. • Short-circuit the pipeline if the request is not authorized.	IAuthorizationFilter IAsyncAuthorizationFilter
Resource Filters	• Run after authorization. • OnResourceExecuting runs code before the rest of the filter pipeline. For example • OnResourceExecuted runs code after the rest of the pipeline has completed.	IResourceFilter IAsyncResourceFilter
Action Filters	• Run immediately before and after an action method is called. • Can change the arguments passed into an action. • Can change the result returned from the action. • Are not supported in Razor Pages.	IActionFilter IAsyncActionFilter
Endpoint Filters	• Run immediately before and after an action method is called. • Can change the arguments passed into an action. • Can change the result returned from the action. • Are not supported in Razor Pages. • Can be invoked on both actions and route handler-based endpoints.	IEndpointFilter
Exception Filters	• Apply global policies to unhandled exceptions that occur before the response body has been written to. • Run after model binding and action filters, but before the action result is executed. • Run only if an unhandled exception occurs during action execution or action result execution. • Do not run for exceptions thrown during middleware execution, routing, or model binding.	IExceptionFilter IAsyncExceptionFilter
Result Filters	• Run immediately before and after the execution of action results. • Run only when the action method executes successfully. • Are useful for logic that must surround view or formatter execution.	IResultFilter IAsyncResultFilter

Filters Implementation in ASP.NET Core?

Filters support both synchronous and asynchronous implementations through different interface definitions.

Synchronous filters run before and after their pipeline stage.
For example:

- OnActionExecuting is called before the action method is called.
- OnActionExecuted is called after the action method returns.

```
public class SampleActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        // Do something before the action executes.
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        // Do something after the action executes.
    }
}
```

Filters Implementation in ASP.NET Core?

Asynchronous filters define an On-Stage-ExecutionAsync method. For example, OnActionExecutionAsync:

```
public class SampleAsyncActionFilter : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(
        ActionExecutingContext context, ActionExecutionDelegate next)
    {
        // Do something before the action executes.
        await next();
        // Do something after the action executes.
    }
}
```

SampleAsyncActionFilter has an ActionExecutionDelegate, next, which executes the action method.

Filter Scope in ASP.NET Core

1. Global Level (applies to all actions)

```
// Program.cs
builder.Services.AddControllers(options =>
{
    options.Filters.Add(new GlobalActionFilter());
});
```

Filter Scope in ASP.NET Core

2. Controller Level (applies to all actions in a controller)

```
[CustomActionFilter]
public class HomeController : Controller
{
    // All actions in this controller will use the filter
}
```

Filter Scope in ASP.NET Core

3. Action Level (applies to specific actions)

```
public class HomeController : Controller
{
    [CustomActionFilter]
    public IActionResult Index()
    {
        return View();
    }
}
```

Default order of execution

- The before code of global filters.
  - The before code of controller filters.
    - The before code of action method filters.
    - The after code of action method filters.
  - The after code of controller filters.
- The after code of global filters.

Differences Between Filters and Middleware

Aspect	Filters	Middleware
Scope of Application (Where)	Applied to **controllers or individual action methods**	Applied **globally** to the entire request pipeline
Execution Point (When)	Execute **inside the MVC framework pipeline**, before or after an action method	Execute **earlier in the request pipeline**, before the request reaches MVC
Purpose (Why)	Handle **MVC-specific concerns** such as authorization, validation, caching, logging, exception handling	Handle **cross-cutting, application-wide concerns** like routing, authentication, logging, compression, security
Configuration (How)	Configured using **attributes** ([Authorize], [ExceptionFilter], etc.) or **globally registered** in Program.cs	Configured and ordered in **Program.cs** using UseMiddleware() and Map()
Execution Order	Determined by **filter type and scope** (authorization → resource → action → result → exception)	Determined strictly by the **order of registration** in the pipeline
Framework Dependency	**Tightly coupled** to ASP.NET Core MVC	**Framework-agnostic** (not tied to MVC)
Access to MVC Context	Has direct access to **ActionContext, ModelState, Result**	No direct access to MVC concepts
Short-Circuiting	Can stop action execution (e.g., authorization filters)	Can stop request processing by not calling next()

TypeFilter vs ServiceFilter

TypeFilter

- Creates the filter using dependency injection
- Automatically resolves constructor dependencies
- Does NOT require the filter itself to be registered in DI

ServiceFilter

- Gets the filter directly from DI container
- Requires manual registration
- If you don't → runtime exception.

TypeFilter vs ServiceFilter

Feature	TypeFilter	ServiceFilter
Needs DI registration	No	Yes
Uses ActivatorUtilities	Yes	No
Good for quick usage	Yes	Not ideal
Best for reusable/global services	Sometimes	Yes

When Should You Use Each?

Use TypeFilter when:

- You don’t want to register filter in DI
- It’s used only in few places
- Quick & simple

Use ServiceFilter when:

- You already registered the filter
- It’s used widely
- You want lifetime control (Scoped/Singleton)

Thank you